UNIT – IV

Applets & Event Handling

Applets: Concepts of Applets, life cycle of an applet, types of applets, creating applets, passing parameters to applets, The AWT class hierarchy, user interface components labels, button, Text components

Event Handling: Events, Delegation event model, handling mouse and keyboard events, Adapter classes, inner classes, Compare basic AWT components with swing components, More user interface components-canvas, scrollbars, check box, choices, lists panels–scroll pane, dialogs, menu bar, layout managers.

Applet Security Policies–Anonymous Inner classes a Short–cut to Event Handling–Java Utilities (java.util Package)

APPLETS

A Java applet is a special kind of Java program that a browser enabled with Java technology can download from the internet and run. An applet is typically embedded inside a web page and runs in the context of a browser. An applet must be a subclass of the **java.applet.Applet** class. The Applet class provides the standard interface between the applet and the browser environment.

The **Applet** class is contained in the **java.applet** package.**Applet** contains several methods that give you detailed control over the execution of your applet.

In addition,**java.applet** package also defines three interfaces: **AppletContext**, **AudioClip**, and **AppletStub**.

Applet Basics:

All applets are subclasses of **Applet**. Thus, all applets must import **java.applet**. Applets must also import **java.awt**. **AWT** stands for the Abstract Window Toolkit. Since all applets run in a window, it is necessary to include support for that window by importing java.awt package.

Applets are not executed by the console-based Java run-time interpreter. Rather, they are executed by either a Web browser or an applet viewer.

Execution of an applet does not begin at **main( ).** Output to your applet's window is not performed by **System.out.println( )**. Rather, it is handled with various AWT methods, such as **drawString( )**, which outputs a string to a specified X,Y location. Input is also handled differently than in an application.

Once an applet has been compiled, it is included in an HTML file using theAPPLET tag. The applet will be executed by a Java-enabled web browser when it encounters the APPLET tag within the HTML file.
To view and test an applet more conveniently, simply include a comment at the head of your Java source code file that contains the APPLET tag.
Here is an example of such a comment:

```
/*
<applet code="MyApplet" width=200 height=60>
</applet>
*/
```

This comment contains an APPLET tag that will run an applet called **MyApplet** in a window that is 200 pixels wide and 60 pixels high. Since the inclusion of an APPLET command makes testing applets easier, all of the applets shown in this tutorial will contain the appropriate APPLET tag embedded in a comment.

The Applet Class:

Applet extends the AWT class **Panel**. In turn, **Panel** extends **Container**, which extends **Component**. These classes provide support for Java's window-based, graphical interface. Thus, **Applet** provides all of the necessary support for window-based activities

Applet Architecture:

An applet is a window-based program. As such, its architecture is different from the so-called normal, console-based programs .

First, applets are event driven. it is important to understand in a general way how the event-driven architecture impacts the design of an applet.

Here is how the process works. An applet waits until an event occurs. The AWT notifies the applet about an event by calling an event handler that has been provided by the applet. Once this happens, the applet must take appropriate action and then quickly return control to the AWT.

An Applet Skeleton:

Four methods—**init( )**, **start( )**, **stop( )**, and **destroy( )**—aredefined by **Applet**. Another, **paint( )**, is defined by the AWT **Component** class. All applets must import **java.applet**. Applets must also import **java.awt.**

These five methods can be assembled into the skeleton shown here:

```
// An Applet skeleton.
import java.awt.*;
import java.applet.*;
/*
<applet code="AppletSkel" width=300 height=100>
</applet>
*/
public class AppletSkel extends Applet
{
  // Called first.
  public void init()
  {
    // initialization
  }
 /* Called second, after init(). Also called whenever
     the applet is restarted. */
 public void start()
{
    // start or resume execution
}
 // Called when the applet is stopped.
 public void stop()
{
   // suspends execution
```

```
  }
  /* Called when applet is terminated. This is the last
  method executed. */
  public void destroy()
 {
    // perform shutdown activities
 }
  // Called when an applet's window must be restored.
  public void paint(Graphics g)
 {
    // redisplay contents of window
 }
}
```

Applet Initialization and Termination:

It is important to understand the order in which the various methods shown in theskeleton are called. When an applet begins, the AWT calls the following methods, in this sequence:

1. **init( )**
2. **start( )**
3. **paint( )**

When an applet is terminated, the following sequence of method calls takes place:

1. **stop( )**
2. **destroy( )**

init( ):init( ) method is called once—the first time an applet is loaded. The **init( )** method is the first method to be called. This is where you should initialize variables.

start(): The **start( )** method is called after **init( )**. It is also called to restart an applet after it has been stopped(i.e start() method is called every time, the applet resumes execution).

Paint(): The **paint( )** method is called each time your applet's output must be redrawn. This situation can occur for several reasons. For example, the window in which the applet is running may be overwritten by another window and then uncovered. Or the applet window may be minimized and then restored. **paint( )** is also called when the applet begins execution. Whatever the cause, whenever the applet must redraw its output, **paint( )** is called. The **paint( )** method has one parameter of type **Graphics**.

stop( ): The stop() method is called when the applet is stopped(i.e for example ,when the applet is minimized the stop method is called).

destroy( ): The **destroy( )** method is called when the environment determines that your applet needs to be removed completely from memory(i.e destroy() method is called when the applet is about to terminate).The **stop( )** method is always called before **destroy( )**.

Simple Applet programe:

SimpleApplet.java

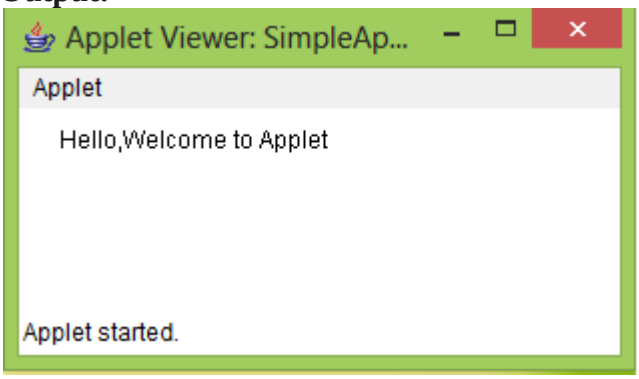
```
import java.awt.*;
import java.applet.*;
/*
<applet code="SimpleApplet" width=300 height=100>
</applet>
*/

public class SimpleApplet extends Applet
{
   String msg="";
   // Called first.
  public void init()
  {
     msg="Hello";
  }
   /* Called second, after init().
   Also called       whenever  the applet is restarted. */
  public void start()
  {
     msg=msg+",Welcome to Applet";
  }

  // whenever the applet must redraw its output, paint( ) is called.
  public void paint(Graphics g)
  {

     g.drawString(msg,20,20);
  }
}
```

Output:



How To Run an Applet Programe:

There are two ways in which you can run an applet:
■ Executing the applet within a Java-compatible Web browser.
■ Using an applet viewer, such as the standard SDK tool, **appletviewer**. An applet viewer executes your applet in a window. This is generally the fastestand easiest way to test your applet.

Using an applet viewer to run applet(demonstrates you to run SimpleApplet.java):

Place the applet tag in comments in java source code.
Note:Code attribute value must be equal to name of class which extends Applet class.

Compiling: javac SimpleApplet.java
Run: AppletViewer SimpleApplet.java

Executing the applet within a Java-compatible Web browser(demonstrates you to run SimpleApplet.java):

Compiling: javac SimpleApplet.java
Create an Html file and embeded Applet tag in html file.
Attributes in applet tag:
Code(attribute):specify name of applet class to load into browser.
Width(attribute):width of an applet.
Height(attribute):height of an applet.

SimpleApplet.html

```
<html>
  <body>
    <applet code="SimpleApplet" width=300 height=100></applet>
  </body>
</html>
```

When you open SimpleApplet.html , SimpleApplet.class applet is loaded into browser.

Note: The Browser must be java enabled to load applet programe.

Simple Applet Display Methods:

As we've mentioned, applets are displayed in a window and they use the AWT to perform input and output.To output a string to an applet, use **drawString( )**, which is a member of the **Graphics** class.Graphics class is defined in **java.awt** package.

void drawString(String *message*, int *x*, int *y*)

Here, *message* is the string to be output and x and y are x-coordinate ,y-coordinate respectively. In a Java window, the upper-left corner is location 0,0.

To set the background color of an applet's window, use **setBackground( )**. To set the foreground color (the color in which text is shown, for example), use **setForeground( )**. These methods are defined by **Component**, and they have the following general forms:

void setBackground(Color *newColor*)
void setForeground(Color *newColor*)

Here, *newColor* specifies the new color. The class **Color** defines the constants shown here that can be used to specify colors:

Color.black Color.magenta
Color.blue Color.orange
Color.cyan Color.pink
Color.darkGray Color.red
Color.gray Color.white
Color.green Color.yellow
Color.lightGray

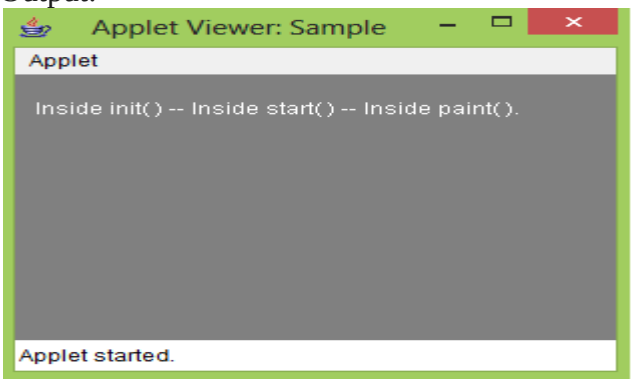
For example, this sets the background color to green and the text color to red:

setBackground(Color.green);
setForeground(Color.red);

Sample.java

```
/* A simple applet that sets the foreground and background colors and outputs a string. */
import java.awt.*;
import java.applet.*;
/*
<applet code="Sample" width=300 height=200>
</applet>
*/
 public class Sample extends Applet
 {
    String msg;
      public void init()
    {
      setBackground(Color.gray);
      setForeground(Color.white);
      msg = "Inside init( ) --";
    }
    // Initialize the string to be displayed.
    public void start()
    {
      msg += " Inside start( ) --";
    }
    // Display msg in applet window.
   public void paint(Graphics g)
   {   msg += " Inside paint( ).";
       g.drawString(msg, 10, 30);
   }
}
```

Output:



Requesting Repainting:

Whenever your applet needs to update the information displayed in its window, it simply calls **repaint( )**.The **repaint( )** method is defined by the AWT. It causes the AWT run-time system to execute a call to your applet's **update( )** method, which, in its default implementation, calls **paint( )**.

The simplest version of **repaint( )** is shown here:

void repaint( )

This version causes the entire window to be repainted. The following version specifies a region that will be repainted:

void repaint(int *left*, int *top*, int *width*, int *height*)

Here, the coordinates of the upper-left corner of the region are specified by *left* and *top*, and the width and height of the region are passed in *width* and *height*. These dimensions are specified in pixels. You save time by specifying a region to repaint.

The other two versions of repaint():

void repaint(long *maxDelay*)

void repaint(long *maxDelay*, int *x*, int *y*, int *width*, int *height*)

Here, *maxDelay* specifies the maximum number of milliseconds that can elapse before **update( )** is called.

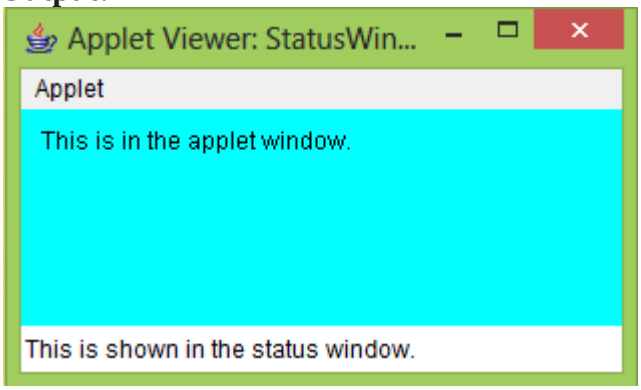
Using the Status Window:

In addition to displaying information in its window, an applet can also output a message to the status window of the browser or applet viewer on which it is running. To do so, call **showStatus( )** with the string that you want displayed.

```
// Using the Status Window.
import java.awt.*;
import java.applet.*;
/*
<applet code="StatusWindow" width=300 height=300>
</applet>
*/
public class StatusWindow extends Applet
{
  public void init()
  {
```

```
    setBackground(Color.cyan);
  }
  // Display msg in applet window.
  public void paint(Graphics g)
  {
    g.drawString("This is in the applet window.", 10, 20);
    showStatus("This is shown in the status window.");
  }
}
```

Output:



The HTML APPLET Tag:

```
< APPLET

 [CODEBASE = codebaseURL]
 CODE = appletFile
 [ALT = alternateText]
 [NAME = appletInstanceName]
 WIDTH = pixels
 HEIGHT = pixels
 [ALIGN = alignment]
 [VSPACE = pixels]
 [HSPACE = pixels]
>
  [< PARAM NAME = AttributeName VALUE = AttributeValue>]
  [< PARAM NAME = AttributeName2 VALUE = AttributeValue>]
  . . .
  [HTML Displayed in the absence of Java]
</APPLET>
```

Let's take a look at each part now.

CODEBASE: CODEBASE is an optional attribute that specifies the base URL of the applet code, which is the directory that will be searched for the applet's executable class file (specified by the CODE tag).
CODE :CODE is a required attribute that gives the name of the file containing your applet's compiled **.class** file. This file is relative to the code base URL of the applet, which is the directory that the HTML file was in or the directory indicated by CODEBASE if set.

ALT :The ALT tag is an optional attribute used to specify a short text message that should be displayed if the browser understands the APPLET tag but can't currently run Java applets.

NAME: NAME is an optional attribute used to specify a name for the applet instance. Applets must be named in order for other applets on the same page to find them by name and communicate with them.

WIDTH AND HEIGHT :WIDTH and HEIGHT are required attributes that give the size (in pixels) of the applet display area.

ALIGN: ALIGN is an optional attribute that specifies the alignment of the applet.The possible values:LEFT, RIGHT, TOP, BOTTOM, MIDDLE, BASELINE, TEXTTOP, ABSMIDDLE,and ABSBOTTOM.

VSPACE AND HSPACE :These attributes are optional. VSPACE specifies the space, in pixels, above and below the applet. HSPACE specifies the space, in pixels, on each side of the applet.

PARAM NAME AND VALUE: The PARAM tag allows you to specify applet specific arguments in an HTML page. Applets access their attributes with the **getParameter( )** method.

Passing Parameters to Applets:

The APPLET tag in HTML allows you to pass parameters to your applet. To retrieve a parameter, use the **getParameter( )** method. It returns the value of the specified parameter in the form of a **String** object. Here is an example that demonstrates passing parameters:

ParamDemo.java

```
import java.awt.*;
import java.applet.*;
/*
<applet code="ParamDemo" width=300 height=300>
<param name=fontName value=Courier>
<param name=fontSize value=14>
<param name=leading value=2>
<param name=accountEnabled value=true>
</applet>
*/

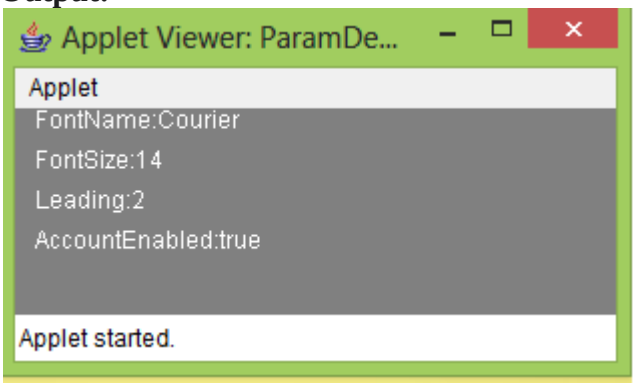
public class ParamDemo extends Applet
{
   String fn=null,fz=null,l=null,ae=null;
   public void init()
   {
     setBackground(Color.gray);
     setForeground(Color.white);
   }
```

```
    public void start()
    {
      fn=getParameter("fontName");
      fz=getParameter("fontSize");
      l=getParameter("leading");
      ae=getParameter("accountEnabled");
      repaint();
    }

    public void paint(Graphics g)
    {
       g.drawString("FontName:"+fn,10,10);
       g.drawString("FontSize:"+fz,10,30);
       g.drawString("Leading:"+l,10,50);
       g.drawString("AccountEnabled:"+ae,10,70);
    }

}
```

Output:



getDocumentBase( ) and getCodeBase( ):

Java will allow the applet to load data from the directory holding the HTML file that started the applet (the *document base*) and the directory from which the applet's class file was loaded (the *code base*). These directories are returned as **URL** objects by **getDocumentBase( )** and **getCodeBase( )**.

Bases.java

```
import java.awt.*;
import java.applet.*;
import java.net.*;
/*<applet code="Bases" width=300 height=50></applet>*/
public class Bases extends Applet
{
  // Display code and document bases.
   public void paint(Graphics g)
  {
     String msg;
     //URL class is defines in java.net package.
     URL url = getCodeBase(); // get code base
```

```
    msg = "Code base: " + url.toString();
    g.drawString(msg, 10, 20);
    url = getDocumentBase(); // get document base
    msg = "Document base: " + url.toString();
    g.drawString(msg, 10, 40);
  }
}
```

Sample output from this program is shown here:



AppletContext and showDocument( ):

To allow your applet to transfer control to another URL, you must use the **showDocument( )** method defined by the **AppletContext** interface.

ACDemo.java

```
import java.awt.*;
import java.applet.*;
import java.net.*;
/*<applet code="ACDemo" width=300 height=50></applet>*/
public class ACDemo extends Applet
{
  URL u;
  public void start()
  {
    AppletContext ac = getAppletContext();
    URL url = getCodeBase(); // get url of this applet
    try
    {
     u=new URL(url+"Test.html");
     ac.showDocument(u);
    }
    catch(MalformedURLException e)
    {
     showStatus("URL not found");
    }
  }
}
```

Explanation: In this programe,the control is transfered from ACDemo.class (i.e applet) to Test.html file.

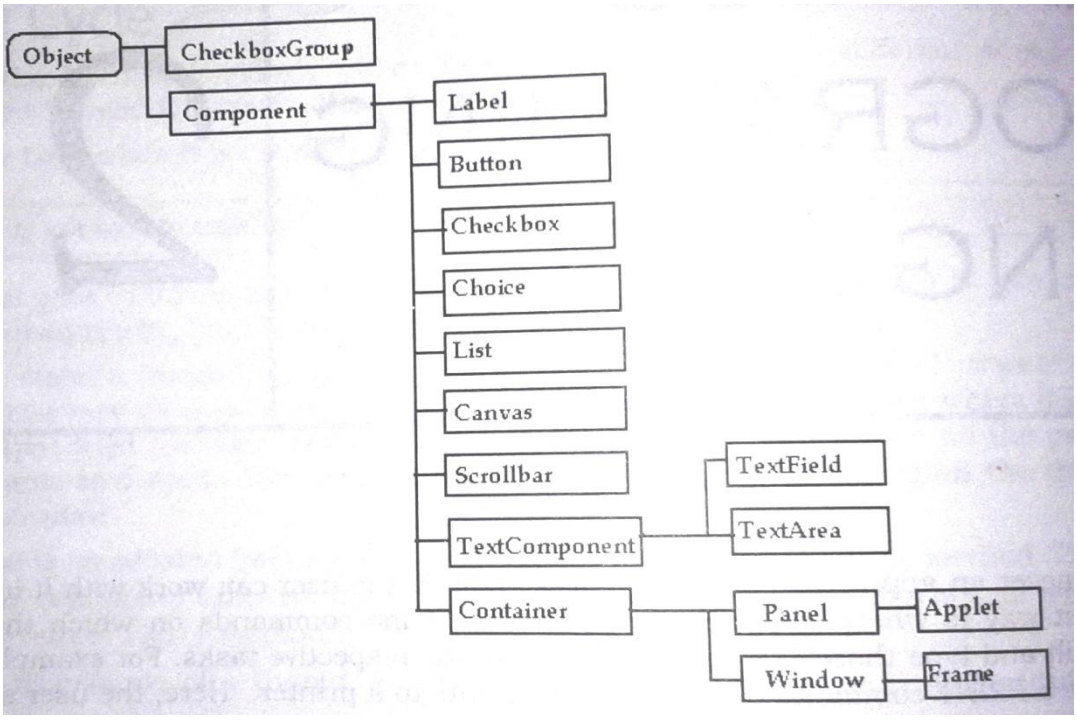
Two Types of Applets:

There are two varieties of applets. The first are those based directly on the **Applet** class. These applets use the Abstract Window Toolkit (AWT) to provide the graphic user interface (or use no GUI at all). This style of applet has been available since Java was first created.

The second type of applets are those based on the Swing class **JApplet**. Swing applets use the Swing classes to provide the GUI. Swing offers a richer and often easier-to-use user interface than does the AWT. Thus, Swing-based applets are now the most popular. **JApplet** inherits **Applet**, all the features of **Applet** are also available in **Japplet**.

AWT (Abstract Window Toolkit):

AWT represents a class library to develop applications using GUI. The **java.awt** package consists of classes and interfaces to develop GUIs.

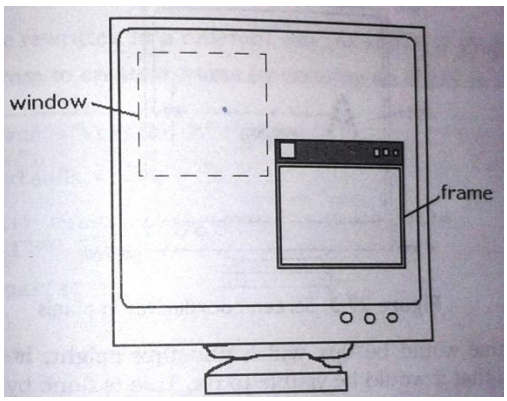


Component: A component represents an object which is displayed pictorially on the screen and interacts with the user.

Ex. Button, TextField, TextArea

Container: A Container is a subclass of Component; it has methods that allow other components to be nested in it. A container is responsible for laying out (that is positioning) any component that it contains. It does this with various layout managers.

Panel: Panel class is a subclass of Container and is a super class of Applet. When screen output is redirected to an applet, it is drawn on the surface of the Panel object. In, essence panel is a window that does not contain a title bar, menu bar or border.

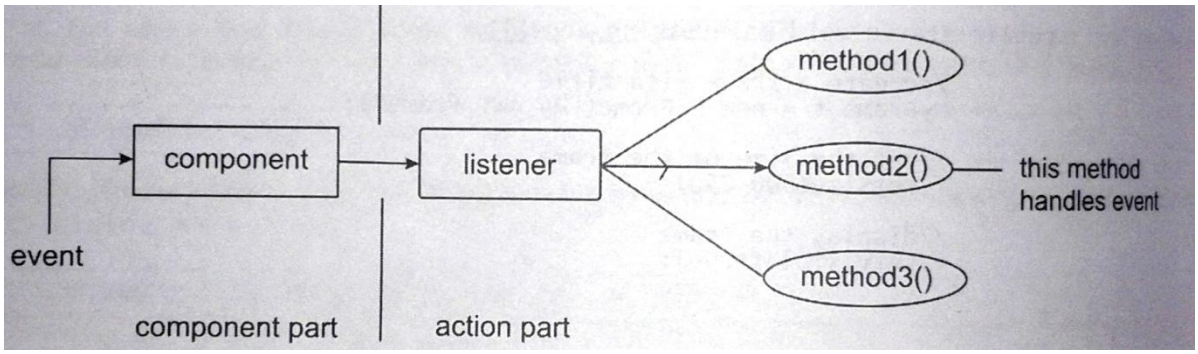


Window: A window represents a rectangular area on the screen without any borders or title bar. The Window class create a top-level window.

Frame: It is a subclass of Window and it has title bar, menu bar, border and resizing windows.

Delegation Event Model:

The modern approach (from version 1.1 onwards) to handle events is based on the delegation event model. Its concept is quite simple: a source generates an event and sends it to one or more listeners.



In this scheme, the listener simply waits until it receives an event. Once an event is received, the listener processes the event and then returns. The advantage of this design is that the application logic that processes events is cleanly separated from the user interface logic that generates those events.

A user interface element is able to "delegate" the processing of an event to a separate piece of code. In the delegation event model, listeners must register with a source in order to receive an event notification. This provides an important benefit: notifications are sent only to listeners that want to receive them.

Events: An *event* is an object that describes a state change in a source. It can be generated as a consequence of a person interacting with the elements in a GUI. Some of the activities that cause events to be generated are pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse.

Event Sources: A source is an object that generates an event. Generally sources are components. Sources may generate more than one type of event.

A source must register listeners in order for the listeners to receive notifications about a specific type of event. Each type of event has its own registration method. Here is the general form:

public void addTypeListener (TypeListener el)

Here, Type is the name of the event, and el is a reference to the event listener. For example, the method that registers a keyboard event listener is called addKeyListener( ).

A source must also provide a method that allows a listener to unregister an interest in a specific type of event. The general form of such a method is this:

public void removeTypeListener(TypeListener el)

Event Listeners: A listener is an object that is notified when an event occurs. It has two major requirements.

1.It must have been registered with one or more sources to receive notifications about specific types of events.

2. It must implement methods to receive and process these notifications.

The methods that receive and process events are defined in a set of interfaces found in *java.awt.event* package.

Sources of Events:

Event Source	Description
Button	Generates action events when the button is pressed.
Check box	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked;
Menu item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scroll bar	Generates adjustment events when the scroll bar is manipulated.
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Event Classes and Listener Interfaces:

The java.awt.event package provides many event classes and Listener interfaces for event handling. At the root of the Java event class hierarchy is **EventObject**, which is in **java.util**. It is the super class for all events. It's one constructor is shown here:

EventObject(Object *src*) - Here, *src* is the object that generates this event.

EventObject contains two methods:

Object getSource( ) - Object on which event initially occurred.

String toString( ) - toString( ) returns the string equivalent of the event.

The class **AWTEvent**, defined within the **java.awt** package, is a subclass of **EventObject**. It is the superclass (either directly or indirectly) of all AWT-based events used by the delegation event model. Its **getID( )** method can be used to determine the type of the event. The signature of this method is shown here:

int getID( )

.

The package **java.awt.event** defines many types of events that are generated by various user interface elements

Event Class	Description	Listener Interface
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.	ActionListener
AdjustmentEvent	Generated when a scroll bar is manipulated.	AdjustmentListener
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.	ComponentListener
ContainerEvent	Generated when a component is added to or removed from a container.	ContainerListener
FocusEvent	Generated when a component gains or losses keyboard focus.	FocusListener
InputEvent	Abstract super class for all component input event classes.	
ItemEvent	Generated when a check box or list item is clicked	ItemListener
KeyEvent	Generated when input is received from the keyboard.	KeyListener
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.	MouseListener and MouseMotionListener
TextEvent	Generated when the value of a text area or text field is changed.	TextListener
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.	WindowListener

Useful Methods of Component class:

Method	Description
public void add(Component c)	inserts a component.
public void setSize(int width,int height)	sets the size (width and height) of the component.
public void setLayout(LayoutManager m)	defines the layout manager for the component.
public void setVisible(boolean status)	changes the visibility of the component, by default false.

The ActionEvent Class:

An ActionEvent is generated when a button is pressed, a list item is double-clicked, or a menu item is selected.

The ActionEvent class defines four integer constants that can be used to identify any modifiers associated with an action event: ALT_MASK, CTRL_MASK, META_MASK (Ex. Escape), and SHIFT_MASK.

ActionEvent has these three constructors:

- ActionEvent(Object src, int type, String cmd)
- ActionEvent(Object src, int type, String cmd, int modifiers)
- ActionEvent(Object src, int type, String cmd, long when, int modifiers)

You can obtain the command name for the invoking ActionEvent object by using the getActionCommand( ) method, shown here:

String getActionCommand( ) -Returns the command string associated with this action

The AdjustmentEvent Class:

An **AdjustmentEvent** is generated by a scroll bar. There are five types of adjustment events.

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

The ComponentEvent Class:

A **ComponentEvent** is generated when the size, position, or visibility of a component is changed. There are four types of component events. The **ComponentEvent** class defines integer constants that can be used to identify them:

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

ComponentEvent is the superclass either directly or indirectly of **ContainerEvent**, **FocusEvent**, **KeyEvent**, **MouseEvent**, and **WindowEvent**, among others.

The **getComponent( )** method returns the component that generated the event. It is shown here:

Component getComponent( )

The ContainerEvent Class:

A **ContainerEvent** is generated when a component is added to or removed from a container. There are two types of container events. The **ContainerEvent** class defines constants that can be used to identify them: **COMPONENT_ADDED** and **COMPONENT_REMOVED**.

The FocusEvent Class:

A **FocusEvent** is generated when a component gains or loses input focus. These events are identified by the integer constants **FOCUS_GAINED** and **FOCUS_LOST**.

The InputEvent Class:

The abstract class **InputEvent** is a subclass of **ComponentEven**t and is the super class for component input events. Its subclasses are **KeyEvent** and **MouseEvent**.

InputEvent defines several integer constants that represent any modifiers, such as the control key being pressed, that might be associated with the event. Originally, the InputEvent class defined the following eight values to represent the modifiers:

ALT_MASK	**ALT_GRAPH_MASK**	**BUTTON2_MASK**	**BUTTON3_MASK**
BUTTON1_MASK	**CTRL_MASK**	**META_MASK**	**SHIFT_MASK**

However, because of possible conflicts between the modifiers used by keyboard events and mouse events, and other issues, the following extended modifier values were added:

ALT_DOWN_MASK	**ALT_GRAPH_DOWN_MASK**	**BUTTON1_DOWN_MASK**
BUTTON2_DOWN_MASK	**BUTTON3_DOWN_MASK**	**CTRL_DOWN_MASK**
META_DOWN_MASK	**SHIFT_DOWN_MASK**	

The KeyEvent Class

A KeyEvent is generated when keyboard input occurs. There are three types of key events, which are identified by these integer constants: **KEY_PRESSED, KEY_RELEASED,** and **KEY_TYPED.**

The first two events are generated when any key is pressed or released. The last event occurs only when a character is generated. Remember, not all key presses result in characters. For example, pressing shift does not generate a character.

There are many other integer constants that are defined by KeyEvent. For example, VK_0 through VK_9 and VK_A through VK_Z define the ASCII equivalents of the numbers and letters.

The MouseEvent Class:

There are eight types of mouse events. The **MouseEvent** class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse
MOUSE_DRAGGED	The user dragged the mouse
MOUSE_ENTERED	The mouse entered a component
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

Two commonly used methods in this class are **getX( )** and **getY( )**. These return the X and Y coordinates of the mouse within the component when the event occurred. Their forms are shown here:

int getX( )
int getY( )

The TextEvent Class:

Instances of this class describe text events. These are generated by text fields and text areas when characters are entered by a user or program. TextEvent defines the integer constant **TEXT_VALUE_CHANGED.**

The WindowEvent Class:

The **WindowEvent** class defines integer constants that can be used to identify different types of events:

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window was iconified.
WINDOW_ICONIFIED	The window gained input focus.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.

EventListener Interfaces:

An event listener registers with an event source to receive notifications about the events of a particular type. Various event listener interfaces defined in the `java.awt.event` package are given below:

Interface	Description
ActionListener	Defines the actionPerformed() method to receive and process action events. *void actionPerformed(ActionEvent ae)*
MouseListener	Defines five methods to receive mouse events, such as when a mouse is clicked, pressed, released, enters, or exits a component *void mouseClicked(MouseEvent me)* *void mouseEntered(MouseEvent me)* *void mouseExited(MouseEvent me)* *void mousePressed(MouseEvent me)* *void mouseReleased(MouseEvent me)*
MouseMotionListener	Defines two methods to receive events, such as when a mouse is dragged or moved. *void mouseDragged(MouseEvent me)* *void mouseMoved(MouseEvent me)*
AdjustmentListner	Defines the adjustmentValueChanged() method to receive and process the adjustment events. *void adjustmentValueChanged(AdjustmentEvent ae)*
TextListener	Defines the textValueChanged() method to receive and process an event when the text value changes. *void textValueChanged(TextEvent te)*
WindowListener	Defines seven window methods to receive events. *void windowActivated(WindowEvent we)* *void windowClosed(WindowEvent we)* *void windowClosing(WindowEvent we)* *void windowDeactivated(WindowEvent we)* *void windowDeiconified(WindowEvent we)* *void windowIconified(WindowEvent we)* *void windowOpened(WindowEvent we)*
ItemListener	Defines the itemStateChanged() method when an item has been *void itemStateChanged(ItemEvent ie)*
WindowFocusListener	This interface defines two methods: **windowGainedFocus( )** and **windowLostFocus( )**. These are called when a window gains or loses input focus. Their general forms are shown here: *void windowGainedFocus(WindowEvent we)* *void windowLostFocus(WindowEvent we)*
ComponentListener	This interface defines four methods that are invoked when a component is resized, moved, shown, or hidden. Their general forms are shown here: *void componentResized(ComponentEvent ce)* *void componentMoved(ComponentEvent ce)* *void componentShown(ComponentEvent ce)* *void componentHidden(ComponentEvent ce)*

ContainerListener	This interface contains two methods. When a component is added to a container, **componentAdded( )** is invoked. When a component is removed from a container, **componentRemoved( )** is invoked. Their general forms are shown here: *void componentAdded(ContainerEvent ce)* *void componentRemoved(ContainerEvent ce)*
FocusListener	This interface defines two methods. When a component obtains keyboard focus, **focusGained( )** is invoked. When a component loses keyboard focus, **focusLost( )** is called. Their general forms are shown here: *void focusGained(FocusEvent fe)* *void focusLost(FocusEvent fe)*
KeyListener	This interface defines three methods. *void keyPressed(KeyEvent ke)* *void keyReleased(KeyEvent ke)* *void keyTyped(KeyEvent ke)*

Steps to perform Event Handling

Following steps are required to perform event handling:

1. Register the component with the Listener
2. Implement the concerned interface

Registration Methods:

For registering the component with the Listener, many classes provide the registration methods. For example:

- **Button**
  - public void addActionListener(ActionListener a){}
- **MenuItem**
  - public void addActionListener(ActionListener a){}
- **TextField**
  - public void addActionListener(ActionListener a){}
  - public void addTextListener(TextListener a){}
- **TextArea**
  - public void addTextListener(TextListener a){}
- **Checkbox**
  - public void addItemListener(ItemListener a){}
- **Choice**
  - public void addItemListener(ItemListener a){}
- **List**
  - public void addActionListener(ActionListener a){}
  - public void addItemListener(ItemListener a){}
- **Mouse**
  - public void addMouseListener(MouseListener a){}

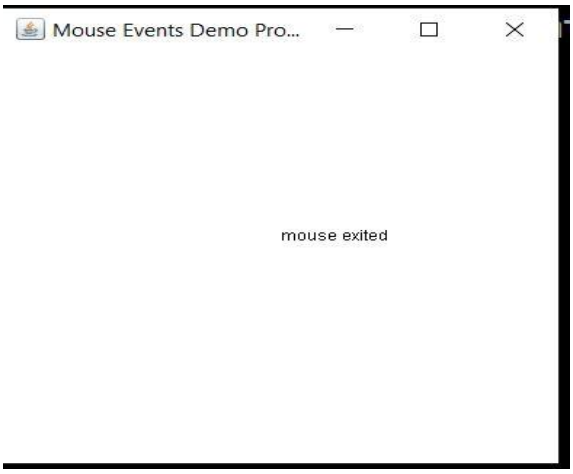
Handling Mouse Events Example Program:

```
// Demonstrate the mouse event handlers.
 import java.awt.*;
import java.awt.event.*;
class MouseDemo extends Frame implements MouseListener
{
	String msg="";
	MouseDemo()
	{
		addMouseListener(this);

	 }
	public void mouseClicked(MouseEvent me)
	{
		msg="mouse clicked"; repaint();
	}
	public void mouseEntered(MouseEvent me)
	{		msg="mouse entered";repaint();
	}
	public void mouseExited(MouseEvent me){
			msg="mouse exited";repaint();
	}
	public void mousePressed(MouseEvent me){
		msg="mouse pressed";repaint();
	}
	public void mouseReleased(MouseEvent me){
		msg="mouse released";repaint();
	}
	public void paint(Graphics g)
	{
		g.drawString(msg,200,200);
	}
}
class MouseEventsExample
{
	public static void main(String arg[])
	{
		MouseDemo d=new MouseDemo();
		d.setSize(400,400);
		d.setVisible(true);
		d.setTitle("Mouse Events Demo Program");

		d.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Output:



Handling Key Board Events:

```
 import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame implements KeyListener
{
        String keystate="Hello PVPSIT";
        String msg="";
        MyFrame()
        {
                addKeyListener(this);
                addWindowListener(new MyWindow() );
        }
        public void keyPressed(KeyEvent ke)
        {
                keystate="Key Pressed";
                msg+=ke.getKeyText( ke.getKeyCode());
                repaint();
        }
        public void keyTyped(KeyEvent ke)
        {
                keystate="Key Typed"; repaint();
                msg=msg+ke.getKeyChar();
        }
        public void keyReleased(KeyEvent ke)
        {
                keystate="Key Released"; repaint();
        }
         public void paint(Graphics g)
        {
                g.drawString(keystate,100,50);
                g.drawString(msg,100,100);
        }
```

```
}
class KeyEventsExample
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setTitle("Key Events Example");
		f.setSize(500,300);
		f.setVisible(true);
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});

	}
}
```

Window Events Example Program:

```
import java.awt.*;
import java.awt.event.*;
 class WindowEx1 extends Frame implements WindowListener
{
	 String msg;
	 WindowEx1(){
		 addWindowListener(this);
	   }
	public void paint(Graphics g)
	{
		g.drawString(msg,150,200);
	}
	public void windowActivated(WindowEvent arg0) {
		msg="activated";
		repaint();
	}
	public void windowClosed(WindowEvent arg0) {
		 msg="closed";                    repaint();
	}
	public void windowClosing(WindowEvent arg0) {
		 System.out.println("closing");
		System.exit(0);
	}
	public void windowDeactivated(WindowEvent arg0) {
		  msg="deactivated" ;
	}
	public void windowDeiconified(WindowEvent arg0) {
		 msg="deiconified" ;                    repaint();
	}
	public void windowIconified(WindowEvent arg0) {
		msg="iconified" ;                repaint();
```

```
        }
        public void windowOpened(WindowEvent arg0) {
                msg="opened" ;                repaint();
        }
}
class WindowExample
{
        public static void main(String[] args) {
                WindowEx1 w=new WindowEx1();
               w.setSize(400,400);
               w.setLayout(null);
                        w.setVisible(true);
        }
}
```

Adapter Classes:

Java provides a special feature, called an *adapter class*, that can simplify the creation of event handlers in certain situations. An adapter class provides an empty implementation of all methods in an event listener interface. Adapter classes are useful when you want to receive and process only some of the events that are handled by a particular event listener interface.

For example,

MouseListener	MouseAdapter
void mouseClicked(MouseEvent me)	*void mouseClicked(MouseEvent me){ }*
void mouseEntered(MouseEvent me)	*void mouseEntered(MouseEvent me) { }*
void mouseExited(MouseEvent me)	*void mouseExited(MouseEvent me) { }*
void mousePressed(MouseEvent me)	*void mousePressed(MouseEvent me) { }*
void mouseReleased(MouseEvent me)	*void mouseReleased(MouseEvent me) { }*

Table: Commonly used Listener Interfaces implemented by Adapter Classes

Adapter Class	Listener Interface
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener

Example Program

```
import java.awt.*;
import java.awt.event.*;
 class WindowEx1 extends Frame
{
        String msg;
```

```
        WindowEx1(){
                addWindowListener(new WA());
         }
 }
class WA extends WindowAdapter
{
        public void windowClosing(WindowEvent we)
        {
                System.exit(0);
        }
}
class WindowAdapterEx
{
        public static void main(String[] args) {
                WindowEx1 w=new WindowEx1();
                w.setSize(400,400);
                w.setLayout(null);
                        w.setVisible(true);
        }
}
```

Inner Classes:

I*nner class* is a class defined within another class, or even within an expression.

Example:

```
         import java.awt.*;
        import java.awt.event.*;
         class WindowEx1 extends Frame
        {
                String msg;
                WindowEx1(){
                        addWindowListener(new WA());
                  }
                class WA extends WindowAdapter
                {
                        public void windowClosing(WindowEvent we)
                        {
                                System.exit(0);
                        }
                }
        }
        class WindowAdapterInner
        {
                public static void main(String[] args) {
                        WindowEx1 w=new WindowEx1();
                        w.setSize(400,400);
                        w.setLayout(null);
                                w.setVisible(true);
```

```
        }
    }
```

Anonymous Inner Classes:

An *anonymous* inner class is one that is not assigned a name.

Example:

```
import java.awt.*;
import java.awt.event.*;
 class WindowEx1 extends Frame
{
    String msg;
    WindowEx1(){
        addWindowListener(new  WindowAdapter()
        {
            public void windowClosing(WindowEvent we)
            {
                System.exit(0);
            }
        });
    }
}
class WindowAdapterAnonymous
{
    public static void main(String[] args) {
        WindowEx1 w=new WindowEx1();
        w.setSize(400,400);
        w.setLayout(null);
            w.setVisible(true);
    }
}
```

Control Fundamentals:

The AWT supports the following types of controls:

- Labels
- Push buttons
- Check boxes
- Choice lists
- Lists
- Scroll bars
- Text Editing

These controls are subclasses of **Component**

Adding and Removing Controls: To include a control in a window, you must add it to the window. To do this, you must first create an instance of the desired control and then add it to a window by calling **add()**, which is defined by **Container**.
The General form is:

Component add(Component *compObj*)

Here, *compObj* is an instance of the control that you want to add. A reference to *compObj* is returned.

Sometimes you will want to remove a control from a window when the control is no longer needed. To do this, call **remove( )**. This method is also defined by **Container**. Here is one of its forms:

void remove(Component *obj*)

Here, *obj* is a reference to the control you want to remove. You can remove all controls by calling **removeAll( )**.

The HeadlessException:

Most of the AWT controls have constructors that can throw a **HeadlessException** when an attempt is made to instantiate a GUI component in a non-interactive environment (such as one in which no display, mouse, or keyboard is present).

Labels:

A *label* is an object of type **Label**, and it contains a string, which it displays. Labels are passive controls that do not support any interaction with the user. **Label** defines the following constructors:

Label( ) throws HeadlessException

Label(String *str*) throws HeadlessException

Label(String *str*, int *how*) throws HeadlessException

The first version creates a blank label. The second version creates a label that contains the string specified by *str*. This string is left-justified. The third version creates a label that contains the string specified by *str* using the alignment specified by *how*. The value of *how* must be one of these three constants: **Label.LEFT**, **Label.RIGHT**, or **Label.CENTER**.

Using Buttons:

A *push button* is a component that contains a label and that generates an event when it is pressed. Push buttons are objects of type **Button**. **Button** defines these two constructors:

Button( ) throws HeadlessException

Button(String *str*) throws HeadlessException

The first version creates an empty button. The second creates a button that contains *str* as a label.

After a button has been created, you can set its label by calling **setLabel( )**. You can retrieve its label by calling **getLabel( )**. These methods are as follows:

void setLabel(String *str*)

String getLabel( )

Here, *str* becomes the new label for the button

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame implements ActionListener
```

```
{
	Button b1,b2,b3;
	MyFrame()
	{
		b1=new Button("Red");
		b2=new Button("Green");
		b3=new Button("Blue");
		add(b1);   add(b2);   add(b3);
		 b1.addActionListener(this);
		b2.addActionListener(this);
		b3.addActionListener(this);
	}
	public void actionPerformed(ActionEvent ae)
	{
		if(ae.getActionCommand()=="Red")
			setBackground(new Color(255,0,0));
		if(ae.getActionCommand()=="Green")
			setBackground(new Color(0,255,0));
		if(ae.getActionCommand()=="Blue")
			setBackground(new Color(0,0,255));
	}
}
class ButtonDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Or

```
import java.awt.*;
import java.awt.event.*;
```

```
class MyFrame extends Frame
{
	Button b1,b2,b3;
	MyFrame()
	{
		b1=new Button("Red");
		b2=new Button("Green");
		b3=new Button("Blue");
		add(b1);   add(b2);   add(b3);

	}

}
class ButtonDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Check Boxes:

A *check box* is a control that is used to turn an option on or off. It consists of a small box that can either contain a check mark or not. There is a label associated with each check box that describes what option the box represents. Check boxes can be used individually or as part of a group. Check boxes are objects of the **Checkbox** class.

Checkbox supports these constructors:

Checkbox( ) throws HeadlessException

Checkbox(String *str*) throws HeadlessException

Checkbox(String *str*, boolean *on*) throws HeadlessException

Checkbox(String *str*, boolean *on*, CheckboxGroup *cbGroup*) throws HeadlessException

Checkbox(String *str*, CheckboxGroup *cbGroup*, boolean *on*) throws HeadlessException

The first form creates a check box whose label is initially blank. The state of the check box is unchecked. The second form creates a check box whose label is specified by *str*. The state of the check box is unchecked. The third form allows you to set the initial state of the check box. If *on* is **true**, the check box is initially checked; otherwise, it is cleared. The fourth and fifth forms create a check box whose label is specified by *str* and whose group is specified by *cbGroup*. If this check box is not part of a group, then *cbGroup* must be **null**. The value of *on* determines the initial state of the check box.

Methods:

boolean getState( ) - To retrieve the current state of a check box
void setState(boolean *on*) - to set the state of a check box
String getLabel( ) – returns the label associated with check box
void setLabel(String *str*) – to set the label

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame  implements  ItemListener
{
	String msg="";
	Checkbox m,f,t;
	MyFrame()
	{
		m=new Checkbox();
		m.setLabel("Male");
		f=new Checkbox("Female",true);
		t=new Checkbox("Transzender");


		add(m); add(f); add(t);
		m.addItemListener(this);
		f.addItemListener(this);
		t.addItemListener(this);
	}
	public void itemStateChanged(ItemEvent ie)
	{
		repaint();
	}
	public void paint(Graphics g)
	{
		msg="Current State";
		g.drawString(msg,150,150);

		msg="Male (True/False)"+m.getState();
		g.drawString(msg,150,200);
```

```
            msg="Female (True/False)"+f.getState();
            g.drawString(msg,150,250);

            msg="Transzender (True/False)"+t.getState();
            g.drawString(msg,150,300);
        }
}
class Demo
{
        public static void main(String arg[])
        {
            MyFrame f=new MyFrame();
            f.setSize(400,400);
            f.setTitle("Event Handling Programs...");
            f.setVisible(true);
            f.setLayout(new FlowLayout());
            f.addWindowListener(new WindowAdapter(){
            public void windowClosing(WindowEvent we)
            {
                System.exit(0);
            }
            });
        }
}
```

Or

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
{
        Checkbox m,f,t;
        MyFrame()
        {
            m=new Checkbox();
            m.setLabel("Male");
            f=new Checkbox("Female",true);
            t=new Checkbox("Transzender");
            add(m); add(f); add(t);
         }
}
class Demo
{
        public static void main(String arg[])
```

```
    {
        MyFrame f=new MyFrame();
        f.setSize(400,400);
        f.setTitle("Event Handling Programs...");
        f.setVisible(true);
        f.setLayout(new FlowLayout());
        f.addWindowListener(new WindowAdapter(){
        public void windowClosing(WindowEvent we)
        {
            System.exit(0);
        }
        });
    }
}
```

CheckboxGroup:

It is possible to create a set of mutually exclusive check boxes in which one and only one check box in the group can be checked at any one time. These check boxes are often called *radio buttons* —only one button can be selected at any one time.

To create a set of mutually exclusive check boxes, you must first define the group to which they will belong and then specify that group when you construct the check boxes. Check box groups are objects of type **CheckboxGroup**.

Only the default constructor is defined, which creates an empty group.

Methods:

Checkbox getSelectedCheckbox( ) - which check box in a group is currently selected
void setSelectedCheckbox(Checkbox *which*) - *which* is the check box that you want to be selected. The previously selected check box will be turned off

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame  implements  ItemListener
{
        String msg="";
        Checkbox m,f,t;
        CheckboxGroup cbg;
        MyFrame()
        {
            cbg=new CheckboxGroup();
            m=new Checkbox("Male",false,cbg);
            f=new Checkbox("Female",false,cbg);
            t=new Checkbox("Transzender",false,cbg);
            cbg.setSelectedCheckbox(f);
```

```
            add(m); add(f); add(t);
            m.addItemListener(this);
            f.addItemListener(this);
            t.addItemListener(this);
      }
      public void itemStateChanged(ItemEvent ie)
      {
            repaint();
      }
      public void paint(Graphics g)
      {
            msg="Current State   :  " +cbg.getSelectedCheckbox();
            g.drawString(msg,150,150);
      }
}
class Demo
{
        public static void main(String arg[])
        {
        MyFrame f=new MyFrame();
        f.setSize(400,400);
        f.setTitle("Event Handling Programs...");
        f.setVisible(true);
        f.setLayout(new FlowLayout());
        f.addWindowListener(new WindowAdapter(){
        public void windowClosing(WindowEvent we)
        {
            System.exit(0);
        }
        });
        }
}
```

Or

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
{
         Checkbox m,f,t;
        CheckboxGroup cbg;
        MyFrame()
        {
```

```
            cbg=new CheckboxGroup();
            m=new Checkbox("Male",false,cbg);
            f=new Checkbox("Female",false,cbg);
            t=new Checkbox("Transzender",false,cbg);
            cbg.setSelectedCheckbox(f);

            add(m); add(f); add(t);
      }
}
class Demo
{
      public static void main(String arg[])
      {
      MyFrame f=new MyFrame();
      f.setSize(400,400);
      f.setTitle("Event Handling Programs...");
      f.setVisible(true);
      f.setLayout(new FlowLayout());
      f.addWindowListener(new WindowAdapter(){
      public void windowClosing(WindowEvent we)
      {
            System.exit(0);
      }
      });
      }
}
```

Choice Controls:

The **Choice** class is used to create a *pop-up list* of items from which the user may choose. **Choice** defines only the default constructor, which creates an empty list. To add a selection to the list, call **add( )**. It has this general form:

void add(String *name*) - *name* is the name of the item being added.

Items are added to the list in the order in which calls to **add( )** occur.

Methods:

String getSelectedItem( ) – returns the item which is currently selected
int getSelectedIndex( ) - returns the index of the item. The first item is at index 0. By default, the first item added to the list is selected.
int getItemCount( ) – returns number of items in the list
void select(int *index*) - to set the currently selected item with index
void select(String *name*) - to set the currently selected item with a string
String getItem(int *index*) – returns the name associated with the index

Example:

```
mport java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame implements ItemListener
{
	Choice c;
	String msg="";
	MyFrame()
	{
		c=new Choice();
		c.add("PVPSIT");
		c.add("VRSEC");
		c.add("BEC");
		c.add("RVR&JC");
		add(c);
		c.addItemListener(this);
	}
	public void itemStateChanged(ItemEvent ie)
	{
		repaint();
	}
	public void paint(Graphics g)
	{
		msg="Current Selection : "+c.getSelectedItem();
		g.drawString(msg, 150,150);

		msg="Selected Index  : "+c.getSelectedIndex();
		g.drawString(msg, 150,200);


		msg="Total No of Items : "+c.getItemCount();
		g.drawString(msg, 150,250);
	}

}
class ChoiceDemo
{
	public static void main(String arg[])
	{
	MyFrame f=new MyFrame();
	f.setSize(400,400);
	f.setTitle("Event Handling Programs...");
	f.setVisible(true);
	f.setLayout(new FlowLayout());
	f.addWindowListener(new WindowAdapter(){
```

```
	public void windowClosing(WindowEvent we)
	{
		System.exit(0);
	}
	});
	}
}

Or
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
{
	Choice c;

	MyFrame()
	{
		c=new Choice();
		c.add("PVPSIT");
		c.add("VRSEC");
		c.add("BEC");
		c.add("RVR&JC");
		add(c);
	}

}
class ChoiceDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
		public void windowClosing(WindowEvent we)
		{
			System.exit(0);
		}
		});
	}
}
```

List:

The **List** class provides a compact, multiple-choice, scrolling selection list. Unlike the **Choice** object, which shows only the single selected item in the menu, a **List** object can be constructed to show any number of choices in the visible window. It can also be created to allow multiple selections.

List provides these constructors:

List( ) throws HeadlessException

List(int *numRows*) throws HeadlessException

List(int *numRows*, boolean *multipleSelect*) throws HeadlessException

The first version creates a **List** control that allows only one item to be selected at any one time. In the second form, the value of *numRows* specifies the number of entries in the list that will always be visible (others can be scrolled into view as needed). In the third form, if *multipleSelect* is **true**, then the user may select two or more items at a time. If it is **false**, then only one item may be selected.

To add a selection to the list, call **add( )**. It has the following two forms:

void add(String *name*)

void add(String *name*, int *index*)

Here, *name* is the name of the item added to the list. The first form adds items to the end of the list. The second form adds the item at the index specified by *index*. Indexing begins at zero. You can specify –1 to add the item to the end of the list.

Example:

```
 // Demonstrate Lists.
import java.awt.*;
import java.awt.event.*;

class MyFrame extends Frame  implements  ActionListener
{
        List l;
        String msg="";
        MyFrame()
        {
                l=new List(2,true);
                l.add("PVPSIT");
                l.add("VRSEC");
                l.add("BEC");
                l.add("RVR&JC");
                add(l);

                l.addActionListener(this);
        }
        public void actionPerformed(ActionEvent ae)
        {
                repaint();
```

```
	}
	public void paint(Graphics g)
	{
		for(int i:l.getSelectedIndexes())
			msg+=i;
		g.drawString(msg,150,150);
		msg="";
		for(String i:l.getSelectedItems())
			msg+=i;
		g.drawString(msg,150,200);


	}
}
class ListDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Or

```
// Demonstrate Lists.
import java.awt.*;
import java.awt.event.*;

class MyFrame extends Frame
{
	List l;
	String msg="";
	MyFrame()
	{
		l=new List(2,true);
```

```
                l.add("PVPSIT");
                l.add("VRSEC");
                l.add("BEC");
                l.add("RVR&JC");
                add(l);

        }
}
class ListDemo
{
        public static void main(String arg[])
        {
                MyFrame f=new MyFrame();
                f.setSize(400,400);
                f.setTitle("Event Handling Programs...");
                f.setVisible(true);
                f.setLayout(new FlowLayout());
                f.addWindowListener(new WindowAdapter(){
                        public void windowClosing(WindowEvent we)
                        {
                                System.exit(0);
                        }
                });
        }
}
```

TextField:

The **TextField** class implements a single-line text-entry area. Text fields allow the user to enter strings and to edit the text using the arrow keys, cut and paste keys, and mouse selections.

TextField is a subclass of **TextComponent**. **TextField** defines the following constructors:

TextField( ) throws HeadlessException
TextField(int *numChars*) throws HeadlessException
TextField(String *str*) throws HeadlessException
TextField(String *str*, int *numChars*) throws HeadlessException

The first version creates a default text field. The second form creates a text field that is *numChars* characters wide. The third form initializes the text field with the string contained in *str*. The fourth form initializes a text field and sets its width.

Methods:

- String getText( ) - To obtain the string currently contained in the text field

- void setText(String *str*) - To set the text, here, *str* is the new string.
- String getSelectedText( ) - returns currently selected text
- void select(int *startIndex*, int *endIndex*) - selects the characters beginning at *startIndex* and ending at *endIndex* –1.
- boolean isEditable( ) – returns boolean value (true/false)
- void setEditable(boolean *canEdit*) - if *canEdit* is **true**, the text may be changed. If it is **false**, the text cannot be altered.
- void setEchoChar(char *ch*) – specified echo character will be displayed in TextField
- boolean echoCharIsSet( ) –returns true or false
- char getEchoChar( ) – returns the echo character

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame implements TextListener
{
	TextField t ;
	String msg="";
	MyFrame()
	{
		Label l=new Label("Enter Name");
		t=new TextField(35);
		 add(l);
		add(t);

		t.addTextListener(this);
	}
	 public void textValueChanged(TextEvent te)
	{
		repaint();
	}
	public void paint(Graphics g)
	{
		msg="Text in the Field : "+t.getText();
		g.drawString(msg,150,150);
		msg="Text in the Field : "+t.getSelectedText();
		g.drawString(msg,150,200);

		 msg="is editable:"+t.isEditable();
		g.drawString( msg ,150,250);

	}
}
```

```
class TextFieldDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Or

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
{
	TextField t ;
	String msg="";
	MyFrame()
	{
		Label l=new Label("Enter Name");
		t=new TextField(35);
		 add(l);
		add(t);
	}
}
class TextFieldDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
```

```
                {
                        System.exit(0);
                }
        });
    }
}
```

TextArea:

Sometimes a single line of text input is not enough for a given task. To handle these situations, the AWT includes a simple multiline editor called **TextArea**. Following are the constructors for **TextArea**:

TextArea( ) throws HeadlessException
TextArea(int *numLines*, int *numChars*) throws HeadlessException
TextArea(String *str*) throws HeadlessException
TextArea(String *str*, int *numLines*, int *numChars*) throws HeadlessException
TextArea(String *str*, int *numLines*, int *numChars*, int *sBars*) throws HeadlessException

Here, *numLines* specifies the height, in lines, of the text area, and *numChars* specifies its width, in characters. Initial text can be specified by *str*. In the fifth form, you can specify the scroll bars that you want the control to have. *sBars* must be one of these values:

SCROLLBARS_BOTH
SCROLLBARS_NONE
SCROLLBARS_HORIZONTAL_ONLY
SCROLLBARS_VERTICAL_ONLY

TextArea is a subclass of **TextComponent**. Therefore, it supports the **getText( )**, **setText( )**, **getSelectedText( )**, **select( )**, **isEditable( )**, and **setEditable( )** methods described in the preceding section.

TextArea adds the following methods:

void append(String *str*) - appends the string specified by *str* to the end of the current
void insert(String *str*, int *index*) - inserts the string passed in *str* at the specified index
void replaceRange(String *str*, int *startIndex*, int *endIndex*) - replaces the characters from *startIndex* to *endIndex*–1, with the replacement text passed in *str*

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
{
      TextArea ta;
      MyFrame()
      {
         ta=new TextArea("PVP Siddhartha Inst. of Technology");

         add(ta);
   }
```

```
}
class TextDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
		public void windowClosing(WindowEvent we)
		{
			System.exit(0);
		}
		});
	}
}
```

Scroll Bars:

Scrollbar control represents a scroll bar component in order to enable user to select from range of values.

Scroll bars are encapsulated by the **Scrollbar** class. **Scrollbar** defines the following constructors:

Scrollbar( ) throws HeadlessException
Scrollbar(int *style*) throws HeadlessException
Scrollbar(int *style*, int *initialValue*, int *thumbSize*, int *min*, int *max*) throws HeadlessException

The first form creates a vertical scroll bar. The second and third forms allow you to specify the orientation of the scroll bar. If *style* is **Scrollbar.VERTICAL**, a vertical scroll bar is created. If *style* is **Scrollbar.HORIZONTAL**, the scroll bar is horizontal. In the third form of the constructor, the initial value of the scroll bar is passed in *initialValue*. The number of units represented by the height of the thumb is passed in *thumbSize*. The minimum and maximum values for the scroll bar are specified by *min* and *max*.

Methods:

void setValues(int *initialValue*, int *thumbSize*, int *min*, int *max*)	If we construct a scroll bar by using one of the first two constructors, then you need to set its parameters by using **setValues()**
int getValue( )	To get the current value
void setValue(int *newValue*)	TO set the current value
int getMinimum( )	To get the minimum value
int getMaximum( )	To get the maximum value

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame implements AdjustmentListener
{
	Scrollbar r,g,b;
	String msg="";
	MyFrame()
	{
		r=new Scrollbar(0,20,10,1,255);
		g=new Scrollbar(1,20,10,1,255);
		b=new Scrollbar(0,20,10,1,255);
		add(r,"South");
		add(g,"East");
		add(b,"North");
		r.addAdjustmentListener(this);
		g.addAdjustmentListener(this);
		b.addAdjustmentListener(this);

	}
	public void adjustmentValueChanged(AdjustmentEvent ae)
	{
		setBackground(new Color(r.getValue(),g.getValue(),b.getValue()));
	}

}
class ScrollbarDemo2
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Scrollbar Programs...");
		f.setVisible(true);
		 f.setLayout(new BorderLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Or

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
```

```
{
	Scrollbar r,g,b;
	String msg="";
	MyFrame()
	{
		r=new Scrollbar(0,20,10,1,255);
		g=new Scrollbar(1,20,10,1,255);
		b=new Scrollbar(0,20,10,1,255);
		add(r );
		add(g);
		add(b);

	}

}
class ScrollbarDemo2
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Scrollbar Programs...");
		f.setVisible(true);
		 f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Layout Manager

A layout manager is a class that is useful to arrange components in a particular manner in container or a frame.

Java soft people have created a LayoutManager interface in java.awt package which is implemented in various classes which provide various types of layouts to arrange the components. The following classes represents the layout managers in Java:

1. FlowLayout
2. BorderLayout
3. GridLayout
4. CardLayout
5. GridBagLayout
6. BoxLayout

To set a particular layout, we should first create an object to the layout class and pass the object to setLayout() method. For example, to set FlowLayout to the container:

```
FlowLayout obj=new FlowLayout();
c. setLayout(obj);  // assume c is container
```

FlowLayout:

FlowLayout is useful to arrange the components in a line one after the other. When a line is filled with components, they are automatically placed in a next line. This is the default layout in applets.

Constructors:

```
FlowLayout( )
FlowLayout(int how)
FlowLayout(int how, int horz, int vert)
```

The first form creates the default layout, which centres components and leaves five pixels of space between each component. The second form lets you specify how each line is aligned. Valid values for how are as follows:

```
FlowLayout.LEFT
FlowLayout.CENTER
FlowLayout.RIGHT
```

The third constructor allows you to specify the horizontal and vertical space left between components in horz and vert, respectively.

Example:

```
 import java.awt.*;
import java.awt.event.*;

class MyFrame extends Frame
{
	List l;
	String msg="";
	MyFrame()
	{
		l=new List(2,true);
		l.add("PVPSIT");
		l.add("VRSEC");
		l.add("BEC");
		l.add("RVR&JC");
		add(l);

	}
}
class ListDemo
{
	public static void main(String arg[])
	{
```

```
                MyFrame f=new MyFrame();
                f.setSize(400,400);
                f.setTitle("Event Handling Programs...");
                f.setVisible(true);
                f.setLayout(new FlowLayout());
                f.addWindowListener(new WindowAdapter(){
                        public void windowClosing(WindowEvent we)
                        {
                                System.exit(0);
                        }
                });
        }
}
```

BorderLayout:

BorderLayout is useful to arrange the components in the four borders of the frame as well as in the centre. The borders are identified with the names of the directions. The top border is specified as 'North', the right side border as 'East', the bottom one as 'South' and the left one as 'West'. The centre is represented as 'Centre'.

Constructors:

- BorderLayout( )
- BorderLayout(int *horz*, int *vert*)

The first form creates a default border layout. The second allows you to specify the horizontal and vertical space left between components in *horz* and *vert*, respectively.

BorderLayout defines the following constants that specify the regions:

BorderLayout.CENTER
BorderLayout.SOUTH
BorderLayout.EAST
BorderLayout.WEST
BorderLayout.NORTH

When adding components, you will use these constants with the following form of **add( )**, which is defined by **Container**:

void add(Component *compObj*, Object *region*)

Here, *compObj* is the component to be added, and *region* specifies where the component will be added.

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
{
    Button b1,b2,b3,b4,b5;
```

```
    MyFrame()
    {
        b1=new Button("East");
        b2=new Button("West");
        b3=new Button("South");
        b4=new Button("North");
        b5=new Button("Center");
        add(b1,"East");
        add(b2,"West");
        add(b3,"South");
        add(b4,"North");
        add(b5,"Center");
    }
}
class BorderDemo
{
    public static void main(String arg[])
    {
        MyFrame f=new MyFrame();
        f.setSize(400,400);
        f.setTitle("Event Handling Programs...");
        f.setVisible(true);

        f.addWindowListener(new WindowAdapter(){
        public void windowClosing(WindowEvent we)
        {
        System.exit(0);
        }
        });
    }
}
```

GridLayout:

GridLayout is useful to divide the container into a 2D grid form that contains several rows and columns. The container is divided into equal-sized rectangle; and one component is placed in each rectangle.

Constructors:

GridLayout( )
GridLayout(int *numRows*, int *numColumns*)
GridLayout(int *numRows*, int *numColumns*, int *horz*, int *vert*)

The first form creates a single-column grid layout. The second form creates a grid layout with the specified number of rows and columns. The third form allows you to specify the horizontal and vertical space left between components in *horz* and *vert*, respectively.

Either *numRows* or *numColumns* can be zero. Specifying *numRows* as zero allows for unlimitedlength columns. Specifying *numColumns* as zero allows for unlimited-length rows.

Example:

```
import java.awt.*;
import java.awt.event.*;
class MyFrame extends Frame
{
	Button b1,b2,b3,b4;
	MyFrame()
	{
		setLayout(new GridLayout(2,2,20,20));
		b1=new Button("PVP");
		b2=new Button("VRSEC");
		b3=new Button("BEC");
		b4=new Button("RVR&JC");
		add(b1 );
		add(b2);
		add(b3);
		add(b4);
	 }
}
class GridDemo
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);

		f.addWindowListener(new WindowAdapter(){
		public void windowClosing(WindowEvent we)
		{
			System.exit(0);
		}
		});
	}
}
```

CardLayout:

A CardLayout object is a layout manager which treats each component as a card. Only one card is displayed at a time, and the container acts as a stack of cards. The first

component added to a CardLayout object is visible component when the container is first displayed.

CardLayout provides these two constructors:

CardLayout( )
CardLayout(int *horz*, int *vert*)

The first form creates a default card layout. The second form allows you to specify the horizontal and vertical space left between components in *horz* and *vert*, respectively.

Use of a card layout requires a bit more work than the other layouts. The cards are typically held in an object of type **Panel**. This panel must have **CardLayout** selected as its layout manager. Finally, you add this pane to the window.

Once these steps are complete, you must provide some way for the user to select between cards. One common approach is to include one push button for each card in the deck. When card panels are added to a panel, they are usually given a name. Thus, most of the time, you will use this form of **add( )** when adding cards to a panel:

void add(Component *panelObj*, Object *name*)
or
void add(Object name, Component *panelObj*)

Here, *name* is a string that specifies the name of the card whose panel is specified by *panelObj*. After you have created a deck, your program activates a card by calling one of the following methods defined by **CardLayout**:

void first(Container *deck*)
void last(Container *deck*)
void next(Container *deck*)
void previous(Container *deck*)
void show(Container *deck*, String *cardName*)

Example:

```
// Demonstrate CardLayout.
import java.awt.*;
import java.awt.event.*;
 class MyFrame extends Frame implements ActionListener
{
        Checkbox winXP, win7, solaris, mac;
        Panel osCards;
        CardLayout cardLO;
        Button Win, Other;

        MyFrame()
        {
                Win = new Button("Windows");
                Other = new Button("Other");
                add(Win);
                add(Other);
```

```
                cardLO = new CardLayout();
                osCards = new Panel();
                osCards.setLayout(cardLO); // set panel layout to card          layout

                winXP = new Checkbox("Windows 7");
                win7 = new Checkbox("Windows 10");
                solaris = new Checkbox("Solaris");
                mac = new Checkbox("Mac OS");

                // add Windows check boxes to a panel
                Panel winPan = new Panel();
                winPan.add(winXP);
                winPan.add(win7);

                // add other OS check boxes to a panel
                Panel otherPan = new Panel();
                otherPan.add(solaris);
                otherPan.add(mac);

                // add panels to card deck panel
                osCards.add(winPan, "Windows");
                osCards.add(otherPan, "Other");
                // add cards to main applet panel
                add(osCards);
                // register to receive action events
                Win.addActionListener(this);
                Other.addActionListener(this);
                // register mouse events
        }
        public void actionPerformed(ActionEvent ae) {
                if(ae.getSource() == Win) {
                                cardLO.show(osCards, "Windows");
                }
                else {
                        cardLO.show(osCards, "Other");
                }
        }
}

class CardLayoutDemo
{
        public static void main(String arg[])
        {
                MyFrame f=new MyFrame();
```

```
            f.setSize(200,200);
            f.setTitle("Event Handling Programs...");
            f.setVisible(true);
            f.setLayout(new FlowLayout());
            f.addWindowListener(new WindowAdapter(){
                  public void windowClosing(WindowEvent we)
                  {
                        System.exit(0);
                  }
            });
      }
}
```

GridBagLayout:

A GridBagLayout class represents grid bag layout manager where the components are arranged in rows and columns. In this layout the component can span more than one row or column and the size of the component can be adjusted to fit in the display area.

When positioning the components by using grid bag layout, it is necessary to apply some constraints or conditions on the components regarding their position, size and place in or around the components etc. Such constraints are specified using GridBagConstrinats class.

In order to create GridBagLayout, we first instantiate the GridBagLayout class by using its only no-argument constructor

```
GridBagLayout layout=new  GridBagLayout();
setLayout(layout);
```

and defining it as the current layout manager.

To apply constraints on the components, we should first create an object to GridBagConstrinats class, as

```
GridBagConstrinats gbc =new GridBagConstrinats();
```

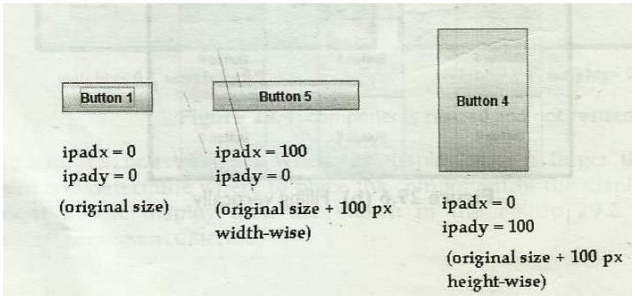
This will create constraints for the components with default value. The other way to specify the constraints is by directly passing their values while creating the GridBagConstrinats as

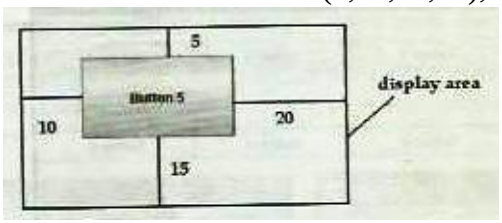
```
GridBagConstrinats gbc= new GridBagConstrinats(
      int gridx, int gridy, int gridwidth, int gridheight, double weightx,
      double  weighty, int    anchor, int fill, Insets insets, int ipadx, int ipady );
```

To set the constraints use setConstraints() method in GridBagConstrinats class and its prototype

```
void setConstraints(Component comp, GridBagConstraints cons);
```

Constraint fields Defined by GridBagConstraints:

Field	Purpose
int anchor	Specifies the location of a component within a cell. The default is GridBagConstraints.CENTER. Others are • GridBagConstraints.EAST • GridBagConstraints.WEST • GridBagConstraints.SOUTH • GridBagConstraints.NORTH • GridBagConstraints.NORTHEAST • GridBagConstraints.NORTHWEST • GridBagConstraints.SOUTHEAST • GridBagConstraints.SOUTHWEST
int gridx	Specifies the X coordinate of the cell to which the component will be added.
int gridy	Specifies the Y coordinate of the cell to which the component will be added.
int gridheight	Specifies the height of component in terms of cells. The default is 1.
int gridwidth	Specifies the width of component in terms of cells. The default is 1.
double weightx	Specifies a weight value that determines the horizontal spacing between cells and the edges of the container that holds them. The default value is 0.0. The greater the weight, the more space that is allocated.
double weighty	Specifies a weight value that determines the vertical spacing between cells and the edges of the container that holds them. The default value is 0.0.
int ipadx	Specifies extra horizontal space that surrounds a component within a cell. The default is 0. 
int ipady	Specifies extra vertical space that surrounds a component within a cell. The default is 0.
int fill	Specifies how a component is resized if the component is smaller than its cell. Valid values are • GridBagConstraints.NONE (the default) • GridBagConstraints.HORIZONTAL • GridBagConstraints.VERTICAL • GridBagConstraints.BOTH.

Insets insets	Small amount of space between the container that holds your components and the window that contains it. Default insets are all zero. Ex. Insets i=new Insets(5,10,20,15); 

Example:

```
 // Use GridBagLayout.
import java.awt.*;
import java.awt.event.*;

class MyFrame extends Frame
{
        Button b1,b2,b3,b4,b5,b6,b7,b8,b9;
        MyFrame()
        {
                GridBagLayout gbag = new GridBagLayout();
                GridBagConstraints gbc = new GridBagConstraints();
                setLayout(gbag);

                b1=new Button("Button 1");
                b2=new Button("Button 2");
                b3=new Button("Button 3");
                b4=new Button("Button 4");
                b5=new Button("Button 5");
                b6=new Button("Button 6");
                b7=new Button("Button 7");
                b8=new Button("Button 8");
                b9=new Button("Button 9");

                gbc.gridx=0;
                gbc.gridy=0;
                gbag.setConstraints(b1,gbc);
                gbc.gridx=1;
                gbc.gridy=0;
                gbag.setConstraints(b2,gbc);
                gbc.gridx=2;
                gbc.gridy=0;
                gbag.setConstraints(b3,gbc);
                gbc.gridx=0;
                gbc.gridy=1;
                gbag.setConstraints(b4,gbc);
                gbc.gridx=1;
                gbc.gridy=1;
                gbag.setConstraints(b5,gbc);
```

```
		gbc.gridx=2;
		gbc.gridy=1;
		gbag.setConstraints(b6,gbc);
		gbc.gridx=0;
		gbc.gridy=2;
		gbag.setConstraints(b7,gbc);
		 gbc.gridx=1;
		gbc.gridy=2;
		gbag.setConstraints(b8,gbc);
		gbc.gridx=2;
		gbc.gridy=2;
		gbag.setConstraints(b9,gbc);

		add(b1);
		add(b2);
		add(b3);
		add(b4);
		add(b5);
		add(b6);
		 add(b7);
		 add(b8);
		  add(b9);
	}
}
class GridBagDemo2
{
	public static void main(String arg[])
	{
		MyFrame f=new MyFrame();
		f.setSize(400,400);
		f.setTitle("Event Handling Programs...");
		f.setVisible(true);
		//f.setLayout(new FlowLayout());
		f.addWindowListener(new WindowAdapter(){
			public void windowClosing(WindowEvent we)
			{
				System.exit(0);
			}
		});
	}
}
```

Java GUI Development (AWT and SWING)

GUI: A graphical user interface (GUI) presents a user-friendly mechanism for interacting with an application. It gives an application a distinctive "look" and "feel." Providing different applications with consistent, intuitive user interface components allows users to be somewhat familiar with an application, so that they can learn it more quickly and use it more productively. GUIs are built from GUI components. These are sometimes called controls or widgets. A GUI component is an object with which the user interacts via the mouse, the keyboard or another form of input, such as voice recognition.

Java GUI APIs: Java has two GUI packages, the original *Abstract Windows Toolkit (**AWT**)* and the newer ***Swing***. When java was introduced, the GUI classes were bundled in a library known as **Abstract Windows Toolkit (AWT).** It is Java's original set of classes for building GUIs. It uses peer components of the OS; It is **heavyweight** and uses the native operating system's window routines so the visual effect is dependent on the run-time system platform. For every platform on which Java runs, the AWT components are automatically mapped to the platform-specific components through their respective agents, known as *peers*. It is not truly portable as it looks different and lays out inconsistently on different OSs. The application's GUI components display differently on each platform. In addition, AWT is adequate for many applications but it is difficult to build an attractive GUI.

Swing: is designed to solve AWT's problems (since Java 2). It is 99% java; It is **lightweight components** as drawing of components is done in java. Swing GUI components allow you to specify a uniform look-and-feel for your application across all platforms. It also lays out consistently on all Oss. It has much bigger set of built-in components and uses AWT event handling.

Swing is built "on top of" AWT, so you need to import AWT and use a few things from it. Swing is bigger and slower. Swing is more flexible and better looking. **Swing** and **AWT** are **incompatible.** Thus, you can use either, but you can't mix them.

Basic components/controls are practically the same in both. For example:

- AWT: Button b = new Button ("OK");
- Swing: JButton b = new JButton("OK");

Swing gives **far more** options for everything (buttons with pictures on them, etc.). AWT classes are contained inside package **java.awt** while Swing classes are located in package **javax.swing.** Below is demonstration of Java GUI ApIs diagrammatically.

Swing vs. AWT

- ✓ First Java GUI library was known as the Abstract Windows Toolkit (AWT).
- ✓ AWT is fine for developing simple graphical user interfaces, but not for complex GUI projects.
- ✓ A newer, more robust, and flexible library is known as Swing components.
- ✓ Swing components are less dependent on the target platform and use less of the native GUI resource.
- ✓ Swing components that don't rely on native GUI are referred to as lightweight components and AWT components are referred to as heavyweight components.
- ✓ To distinguish new Swing component classes from their older AWT counterparts, the Swing GUI component classes are named with a prefixed J.
- ✓ Although AWT components are still supported in Java, it is better to learn to how program using Swing components, because the AWT user- interface components will eventually fade away.

Difference Between AWT and Swing

Definition

AWT is a collection of GUI components (widgets) and other related services required for GUI programming in Java. Swing is a part of Java Foundation Classes (JFC) that is used to create Java-based Front end GUI applications. Hence, this explains the main difference between AWT and Swing in Java.

Type

AWT components are heavyweight while Swing components are lightweight.

Platform Dependency

Another major difference between AWT and Swing in Java is that AWT is platform dependent while Swing is platform independent.

Display

Moreover, AWT does not support a pluggable look and feel. Swing supports a pluggable look and feel. This is also an important difference between AWT and Swing in Java.

Components

Also, Swing has more advanced components than AWT.

Speed

Furthermore, execution of AWT is slower. However, Swing executes faster.

MVC

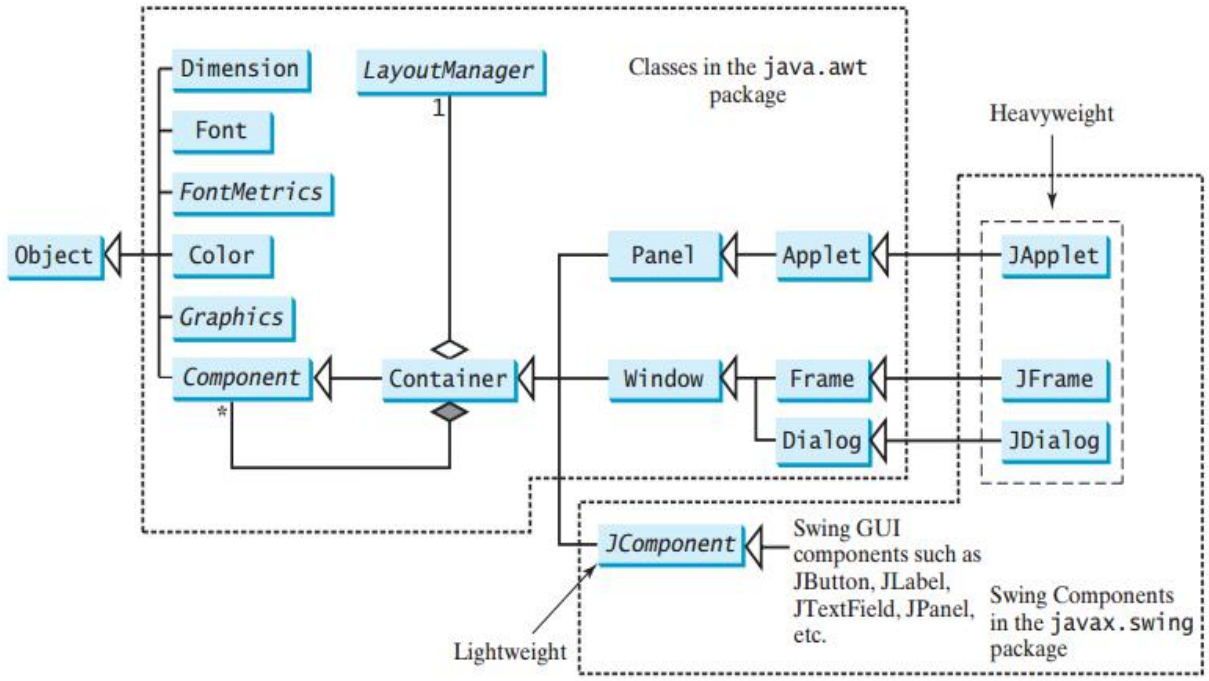
AWT does not support MVC pattern while Swing supports MVC pattern. This is another difference between AWT and Swing.

Memory Space

Moreover, AWT components require more memory space while Swing components do not require much memory space.

Package

The programmer has to import the javax.awt package to develop an AWT-based GUI. However, the programmer has to import javax.swing package to write a Swing application.



Generally, the Java GUI APIs are categorized into three groups. These are **Container class**, **Component classes and Helper class**.

GUI Container classes: A GUI is built by putting components/controls into containers. Thus, container is used to group components. **Frames**, **Panels** and **applets** are examples of containers. Important Container classes are **JFrame**, **JApplet**, **JDialog** and **JPanel.**

Frame: A resizable, movable window with title bar and close button. Usually it contains JPanels. It is a container that holds other Swing user-interface components in Java GUI application.

JPanel: JPanel is a region internal to a JFrame or another JPanel. It is used for grouping components together. JPanel is optionally bounded by a visible border . It lives inside some enclosing Container. Panels can be nested, that is, you can place panels inside a container that includes a panel. In fact, every frame has *at least* one pane, the default "**Content Pane**".

GUI Component classes: GUI Components or Controls (also known as "**widgets**") are the basic user interface elements the user interacts with. These include labels, buttons and text fields. The visual arrangement of the components depends on the container's **layout** (which are part of helper classes).

Here is a list of controls in the javax.swing package.

- **Input Components**
  - Buttons (JButton, JRadioButtons, JCheckBox)

- Text (JTextField, JTextArea)
- Menus (JMenuBar, JMenu, JMenuItem)
- Sliders (JSlider)
- JComboBox (uneditable) (JComboBox)
- List (Jlist)

- **Information Display Components**
  - JLabel
  - Progress bars (JProgressBar)
  - Tool tips (using JComponent's setToolTipText(*s*) method)
- **Choosers**
  - File chooser (JFileChooser)
  - Color chooser (JColorChooser)
- **More complex displays**
  - Tables (JTable)
  - Trees (JTree)

GUI helper classes: Helper classes are used to describe the properties of GUI components. It encompasses the classes for graphics context, colors, fonts and dimension.

Graphics: It is an abstract class that provides a graphical context for drawings strings, lines, and simple shapes.

Color: It deals with the colors of GUI components.

For example, we can specify background colors in components like JFrame and JPanel while we can specify colors of lines, shapes and etc.

Font: It specifies fonts for the text and drawings on GUI components. Font is available on the java.awt package.

Example: panel1.setFont(new Font("Serif",Font.TRUETYPE_FONT + Font.ITALIC + Font.BOLD,20));

LayoutManager: LayoutManager is an interface whose instances specify how components are arranged in a container. There are different types of layouts. FlowLayout, GridLayout and BorderLayout are three examples of the layouts. These layouts are found in the java.awt package of JDK API.

Let us see an example using one of these three layout types.

```
LayoutManager lay_out = new FlowLayout();
JFrame Frame_based_container = new JFrame("I am a container");
Frame_based_container.setLayout(lay_out);
setLayout(new FlowLayout());//default one
setLayout(new FlowLayout(FlowLayout.LEFT));
setLayout(new FlowLayout(FlowLayout.LEFT, 20,30));
```

It should be noted that before adding components to a certain container, the layout must be first set depending on the desired arrangement of the containers or components inside the container.

Nowadays, Java GUI focuses on Swing GUI components and Swing Containers. At the meantime, the helper classes are used from AWT.

Steps in building GUI Application

1. **Creating GUI Containers**: Create a container that could contain components or other containers. We need to create either a frame or an applet to hold the user-interface components.

   Frame: It is an independent window that has decorations such as a border, a title and buttons for closing, minimizing and maximizing the window. Frame is a window that is not contained inside another window. A frame can be moved around on the screen independently of any other GUI windows. Applications with a GUI typically use **at least one frame.** Frame is the basis to contain other user interface components in Java GUI applications.

The **JFrame** class can be used to create windows. The following statement creates a window with title "My First JFrame".

JFrame frame = new Jframe("My First JFrame");

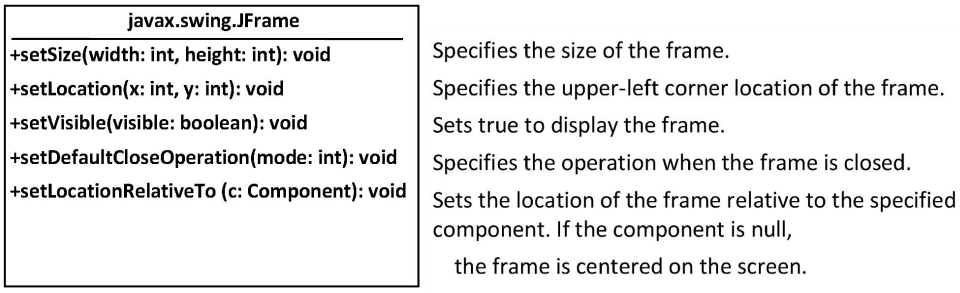


The JFrame class has the following two constructors:

JFrame(): This constructor constructs a new frame with no title and it is initially invisible.

JFrame(String title): This constructor constructs a new, initially invisible frame with specified title.

Some of the methods of JFrame class are described here in class diagram:



setBounds(int x, int y, int width,Int height) : It is another method of JFrame class that encompasses the methods setLocation(int x, int y) and setSize(int width, int height). It specifies the size of the frame and the location of the upperbleft corner. This puts the upper left corner at location (x, y), where x the number of pixels from the left of the screen and y is the number from the top of the screen. height and width are as before.

public void setDefaultCloseOperation(int mode): It is a method of JFrame class that is used to specify one of several options for the close button. Every JFrame instance window has a close button. We can program how the window reacts when this close button is clicked.

Use one of the following constants to specify your choice:

JFrame.EXIT_ON_CLOSE: exit the application.

JFrame.HIDE_ON_CLOSE: Hide the frame, but keep the application running.

JFrame.DO_NOTHING_ON_CLOSE: Ignore the click.

Example: Here is a java code that creates a window/frame with the specified title, location, size, frame visibility and default close operation. Let you go through the code and understand what each line of the code means.

```
import javax.swing.*;
public class JFrameSample {
  public static void main(String[] args) {
   JFrame frame = new JFrame("First JFrame");
   frame.setSize(400, 300);
```

```
frame.setLocationRelativeTo(null);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
```

Here is another way of writing the above code (by extending JFframe):

```
import javax.swing.JFrame;
public class Simple extends JFrame {
  public Simple() {
   setSize(300,200);
   setTitle("First JFrame");
   setLocationRelativeTo(null);
   setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
   setVisible(true);
  }
 public static void main(String[] args) {
   Simple simple = new Simple();
  } }
```

2. Create some more Components (such as buttons and text areas)

JButton: A button is a component that triggers an action event when clicked. A JButton component can be created using one of the following constructors.

- **JButton():** Creates a default button with no text and icon.
- **JButton(Icon icon):** Creates a button with an icon.
- **JButton(String text):** Creates a button with text.
- **JButton(String text, Icon icon):** Creates a button with text and an icon.

JButton class has different methods including the following:

- **public void setText(String text) :** Sets the button's text.
- **public String getText():** Returns the button's text.
- **public void setEnabled(boolean b) :** Enables (or disables) the button.
- **public void setSelectedIcon(Icon selectedIcon):** Sets the selected icon for the button.

- **public boolean isSelected() :** Returns the state of the button. True if the toggle button is selected, false if it's not.

Here two alternative examples that create a JButton component with text “Click Here”.

```
JButton button= new JButton();
button.setText(“Click Here”);
OR
JButton button= new JButton(“Click Here");
```

JLabel: A label is a display area for a short text(a non-editable), an image, or both.

A JLabel component can be created using one of the following constructors:

- **JLabel():** Creates a default label with no text and icon.
- **JLabel(String text):** Creates a label with text.
- **JLabel(Icon icon):** Creates a label with an icon.
- **JLabel(String text, int horizontalAlignment):** Creates a label with a text and the specified horizontal alignment.
- **JLabel(Icon icon, int horizontalAlignment):** Creates a label with an icon and the specified horizontal alignment.
- **JLabel(String text, Icon icon, int horizontalAlignment): C**reates a label with text, an icon, and the specified horizontal alignment.

JLabel component has various methods including the following:

- **public String getText():** Returns a string containing the text in the label component
- **public void setText(String):** Sets the label component to contain the string value
- **public Icon getIcon():** Returns the graphic image (icon) that the label displays.
- **public void setIcon(Icon icon):** Defines the icon this component will display. If the value of icon is null, nothing is displayed.

For example, JLabel can be used as follows.

```
// Create an image icon from image file
 ImageIcon icon = new ImageIcon("image/grapes.gif");

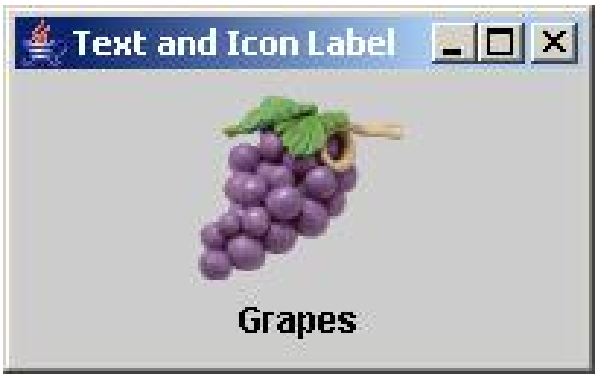
// Create a label with text, an icon, with centered horizontal alignment
 JLabel jlbl = new JLabel("Grapes", icon, SwingConstants.CENTER);
```

// Set label's text alignment and gap between text and icon

jlbl.setHorizontalTextPosition(SwingConstants.CENTER);

jlbl.setVerticalTextPosition(SwingConstants.BOTTOM);

jlbl.setIconTextGap(5);



JTextField: A **text field** is a box that contains a line of text. The user can type text into the box and the program can get it and then use it as data. The program can write the results of a calculation to a text field. Text fields are useful in that they enable the user to enter in variable data (such as a name or a description). **JTextField** is swing class for an editable text display.

JTextField components can be created using one of the following constructors:

- **JTextField()**: it creates a default empty text field with number of columns set to 0.
- **JTextField(int columns)**: it creates an empty text field with the specified number of columns.
- **JTextField(String text)**: it creates a text field initialized with the specified text.
- **JTextField(String text, int columns)**: it creates a text field initialized with the specified text and the column size.

JTextfield class has the following methods:

- **public String getText():** returns the string from the text field.
- **public void setText(String text):** puts the given string in the text field.
- **public void setEditable(boolean editable):** enables or disables the text field to be edited. By default, editable is true.
- **public void setColumns(int): s**ets the number of columns in this text field. The length of the text field is changeable.

- ℘ **public void select(int selectionStart, int selectionEnd):** Selects the text between the specified start and end positions.
- ℘ **public String getSelectedText():** Returns the text value that has been highlighted in the text field.
- ℘ **public void append(String value):** Appends the text value of the string to the already existing text in the component

Here is an example in which JTextField is used.

```
JLabel Fname= new JLabel("First Name");
 JTextField text = new JTextField(10);
JLabel Lname= new JLabel("Last Name");
JTextField text1 = new JTextField("Text1",10);
text.setEditable(false);
add(Fname); add(text); add(Lname); add(text1);
```

Exercise: Draw the resulting GUI of this code.

JPasswordField: Allows the editing of a single line of text where the view indicates something was typed, but does not show the original characters.

JPasswordField component can be created using one of the following constructors:

- **JPasswordField():** Constructs a new JPasswordField, with a default document, null starting text string, and 0 column width
- **JPasswordField(int columns):** Constructs a new empty JPasswordField with the specified number of columns
- **JPasswordField(String text):** Constructs a new JPasswordField initialized with the specified text.
- **JPasswordField(String text, int columns) :** Constructs a new JPasswordField initialized with the specified text and columns.

JTextArea: If you want to let the user enter multiple lines of text, you cannot use text fields unless you create several of them. The solution is to use JTextArea class, which enables the user to enter multiple lines of text.

JTextArea components can be created using one of the following constructors:

- **JTextArea(int rows, int columns):** creates a text area with the specified number of rows and columns.

- **JTextArea(String s, int rows, int columns):** creates a text area with the initial text and the number of rows and columns specified.

JCheckBox: It is a widget that has two states. On and Off. It is a box with a label. If the checkbox is checked, it is represented by a tick in a box. JCheckBox component can be created using one of the following constructors:

- **JCheckBox**() :creates an initially unselected check box button with no text, no icon
- **JCheckBox**(Icon icon) : creates an initially unselected check box with an icon
- **JCheckBox**(Icon icon, boolean selected) : creates a check box with an icon and specifies whether or not it is initially selected.
- **JCheckBox**(String text) : creates an initially unselected check box with text.
- **JCheckBox**(String text,boolean selected): creates a check box with text and specifies whether or not it is initially selected.
- **JCheckBox(String text,Icon icon) : c**reates an initially unselected check box with the specified text and icon.
- **JCheckBox(String text, Icon icon, Boolean selected) :** creates a check box with text and icon, and specifies whether or not it is initially selected.

Example:

JCheckBox red = new JCheckBox("Red", true);

JCheckBox blue = new JCheckBox("Blue");

JCheckBox green = new JCheckBox("Green");

JCheckBox yellow = new JCheckBox("Yellow");



Creating Menu Driven Java Application:

We can create a menu driven application in java from which we can choose an activity.

There are three important swing components for the development of menu driven application. These are **JMenuBar**, **JMenu** and **JMenuItem.**

JMenuBar: It is used to create the menu bar. Menu bar is created using the constructor **JMenubar().** For example, we may create a menu bar as follows.

JMenuBar jmb = new JMenuBar();

Once a menu bar is created, the menu bar is going to be embedded onto the container frame using the method **setJMenuBar(JMenuBar mbr).**

JMenu: It is used to create menu or submenus. The constructor JMenu(String name) can be used to create a menu/submenu. For example, JMenu fileMenu = new JMenu("File").

Once menus are created using JMenu, these menus can be added onto the menu bar using the function **add(JMenu menu).** Assume we have created two menu component file and edit as follow.

JMenu fileMenu = new JMenu("File").

JMenu editMenu = new JMenu("Edit").

We can add these menu components to the menu bar we created earlier as follow

Jmb.add(fileMenu);

Jmb.add(editMenu)

JMenuItem: This component is used to add menu items under menu components. The constructor **JMenuItem(String item)** is used to create such items. Once the items are created the method **add(JMenuItem item)** can be used to add the item to a menu component.

For example, let us create two menu items under the file menu.

JMenuItem menuItemNew = new JMenuItem("New");

JMenuItem menuItemOpen = new JMenuItem("Open");

fileMenu.add(menuItemNew);

fileMenu.add(menuItemOpen);

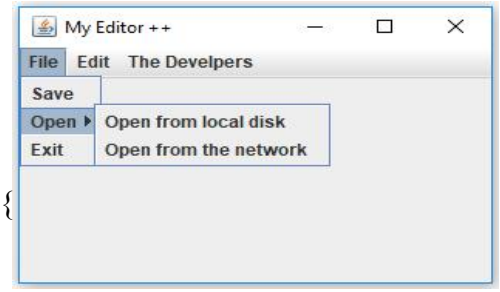
Creating submenu under a menu: It is possible to have a menu under another menu, that is, creating a submenu under a menu. First, the submenu will be created using the **JMenu** constructor. Then, we will use the method **add(JMenu menu)** to add the submenu under a menu. Let u observe the complete code of the following example and notice how a submenu is added to a menu.

Example: consider the following example that creates the output displayed next to the code

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JFrame;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;
import javax.swing.JOptionPane;

public class ClassAtMenuing {
   public static void main(String[] args) {
```

```
        JFrame myEditor = new JFrame("My Editor ++");
        JMenuBar myMenuBar = new JMenuBar();
        myEditor.setJMenuBar(myMenuBar);
        //Create the Menus
        JMenu fileMenu = new JMenu("File");
        JMenu editMenu = new JMenu("Edit");
        JMenu developersMenu = new JMenu("The Develpers");
        //add meu items under File
        JMenuItem itemSave = new JMenuItem("Save");
        JMenuItem closeWindow = new JMenuItem("Exit");
        //JMenuItem itemOpen = new JMenuItem("Open");
        fileMenu.add(itemSave);
         //adding Submen
         JMenu openMenu = new JMenu("Open");
         //items under open
         JMenuItem openFromLocalDisk = new JMenuItem("Open from local disk");
         JMenuItem openFRomNet = new JMenuItem("Open from the network");
         //add the items to the open JMenu
         openMenu.add(openFromLocalDisk);
         openMenu.add(openFRomNet);
         fileMenu.add(openMenu);
         fileMenu.add(closeWindow);
         closeWindow.addActionListener(new ActionListener(){
           @Override
           public void actionPerformed(ActionEvent e) {
              JOptionPane.showMessageDialog(null, "You are closing thed editor");
              myEditor.dispose();
           }
         });
        //add menu into the menubar
        myMenuBar.add(fileMenu);
        myMenuBar.add(editMenu);
        myMenuBar.add(developersMenu);
        myEditor.setBounds(100, 150, 400, 300);
        myEditor.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        myEditor.setVisible(true);
           }
}
```



Java JRadioButton

The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. It is widely used in exam systems or quiz.

It should be added in ButtonGroup to select one radio button only.

Constructor	Description
JRadioButton()	Creates an unselected radio button with no text.
JRadioButton(String s)	Creates an unselected radio button with specified text.
JRadioButton(String s, boolean selected)	Creates a radio button with the specified text and selected status.

Commonly used Constructors:

Commonly used Methods:

Methods	Description
void setText(String s)	It is used to set specified text on button.
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener(ActionListener a)	It is used to add the action listener to this object.

Java JRadioButton Example

```
1. import javax.swing.*;
2. public class RadioButtonExample {
3. JFrame f;
4. RadioButtonExample(){
```

```
5.  f=new JFrame();
6.  JRadioButton r1=new JRadioButton("A) Male");
7.  JRadioButton r2=new JRadioButton("B) Female");
8.  r1.setBounds(75,50,100,30);
9.  r2.setBounds(75,100,100,30);
10. ButtonGroup bg=new ButtonGroup();
11. bg.add(r1);bg.add(r2);
12. f.add(r1);f.add(r2);
13. f.setSize(300,300);
14. f.setLayout(null);
15. f.setVisible(true);
16. }
17. public static void main(String[] args) {
18.     new RadioButtonExample();
19. }
20. }
```

Output:



Java JComboBox

The object of Choice class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu. It inherits JComponent class.

Commonly used Constructors:

Constructor	Description
JComboBox()	Creates a JComboBox with a default data model.
JComboBox(Object[] items)	Creates a JComboBox that contains the elements in the specified array.
JComboBox(Vector<?> items)	Creates a JComboBox that contains the elements in the specified Vector.

Commonly used Methods:

Methods	Description
void addItem(Object anObject)	It is used to add an item to the item list.
void removeItem(Object anObject)	It is used to delete an item to the item list.
void removeAllItems()	It is used to remove all the items from the list.
void setEditable(boolean b)	It is used to determine whether the JComboBox is editable.
void addActionListener(ActionListener a)	It is used to add the ActionListener.
void addItemListener(ItemListener i)	It is used to add the ItemListener.

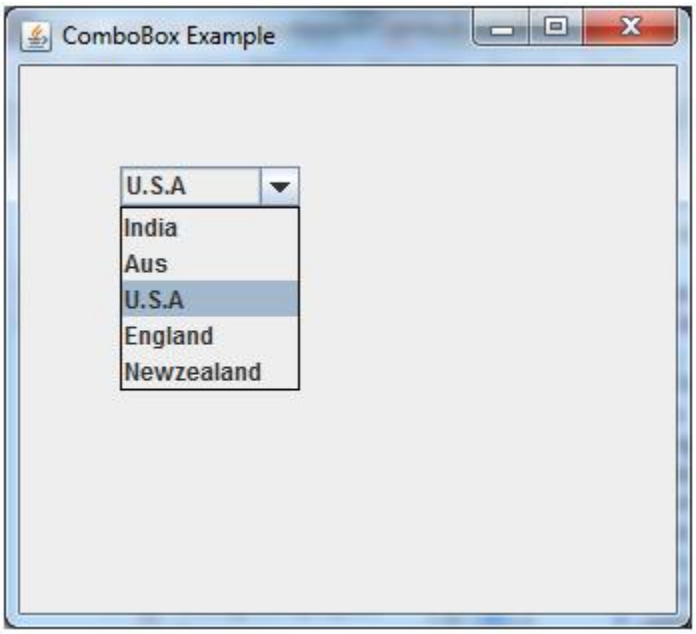
Java JComboBox Example

```
1. import javax.swing.*;
2. public class ComboBoxExample {
3. JFrame f;
4. ComboBoxExample(){
5.     f=new JFrame("ComboBox Example");
6.     String country[]={"India","Aus","U.S.A","England","Newzealand"};
```

Constructor	Description

```
7.      JComboBox cb=new JComboBox(country);
8.      cb.setBounds(50, 50,90,20);
9.      f.add(cb);
10.     f.setLayout(null);
11.     f.setSize(400,500);
12.     f.setVisible(true);
13. }
14. public static void main(String[] args) {
15.     new ComboBoxExample();
16. }
17. }
```

Output:



Java JTable

The JTable class is used to display data in tabular form. It is composed of rows and columns

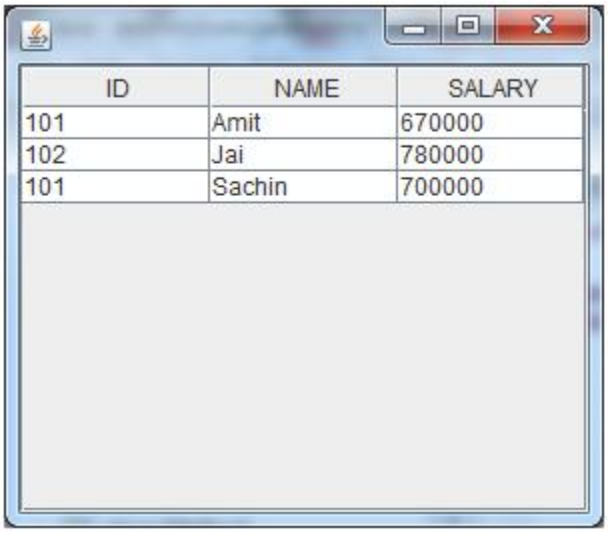
Commonly used Constructors:

JTable()	Creates a table with empty cells.
JTable(Object[][] rows, Object[] columns)	Creates a table with the specified data.

Java JTable Example

```
1. import javax.swing.*;
2. public class TableExample {
3.     JFrame f;
4.     TableExample(){
5.     f=new JFrame();
6.     String data[][]={ {"101","Amit","670000"},
7.                       {"102","Jai","780000"},
8.                       {"101","Sachin","700000"}};
9.     String column[]={"ID","NAME","SALARY"};
10.    JTable jt=new JTable(data,column);
11.    jt.setBounds(30,40,200,300);
12.    JScrollPane sp=new JScrollPane(jt);
13.    f.add(sp);
14.    f.setSize(300,400);
15.    f.setVisible(true);
16. }
17. public static void main(String[] args) {
18.    new TableExample();
19. }
20. }
```

Output:



3. Add your components to your display area: choose a layout manager.

Each JFrame contains a **content pane**. A content pane is an instance of **java.awt.Container**. The GUI components such as buttons are placed in the content pane in a frame. Prior to JDK 1.5, you have to use the **getContentPane()** method in the JFrame class to return the content pane of the frame, and then invoke the content pane's **add** method to place a component into a content pane.

This was cumbersome, JDK 1.5 allows you to place the components to the content pane by invoking a frame's **add** method. This new feature is called **content pane delegation.** Strictly speaking, a component is added into the content pane of the frame. But for simplicity we say that a component is added to a frame. When a component is added to a container, a reference to the component is added to the list of components in the container.

Below are four alternatives to add a button with text "OK" into a frame instance, "frame". We assume that the frame has been instantiated somewhere.

Example 1: Jbutton jbt = new Jbutton("OK");

frame.getContentPane().add(jbt);

Example 2: frame.getContentPane().add(new JButton("OK"));

Example 3: JButton jbt = new Jbutton("OK")

frame.add(jbt);

Example 4: frame.add(new JButton("OK"));

Below is a full java code that adds the button to the frame (following the approach employed by example 3 above)

```
import javax.swing.*;
public class MyFrameWithComponents {
  public static void main(String[] args) {
    JFrame frame = new JFrame("MyFrameWithComponents");
    // Add a button into the frame
    JButton jbtOK = new JButton("OK");
    frame.add(jbtOK);
    frame.setSize(400, 300);
    frame.setVisible(true);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setLocationRelativeTo(null); // New since JDK 1.4
  }
}
```

When you run the above program **MyFrameWithComponents**, the button is always centered in the frame and occupies the entire frame no matter how you resize it. This is because components are put in the frame by the content pane's layout manager, and the default layout manager for the content pane places the button in the **center**. Set layout Manager before adding components. In the next section, you will use several different layout managers to place components in other location as desired.

Layout Management:

Layouts tell Java **where to put components** in containers (**JPanel**, **content pane**, etc). Layout manager is created using LayoutManager class. Every layout manager class implements the **LayoutManager** class. Each layout manager has a different style of positioning components. If you don't specify otherwise, the container will use a default layout manager. Every **panel** (and other container) has a default layout, but it's better to set the layout explicitly for clarity.

Layout managers are set in containers using the **setLayout(LayoutManager)** method.

Example: assume we have a Jframe instance called container. If we are going to add controls inside this container and if we are interested on the flowlayout, we can set as follow.

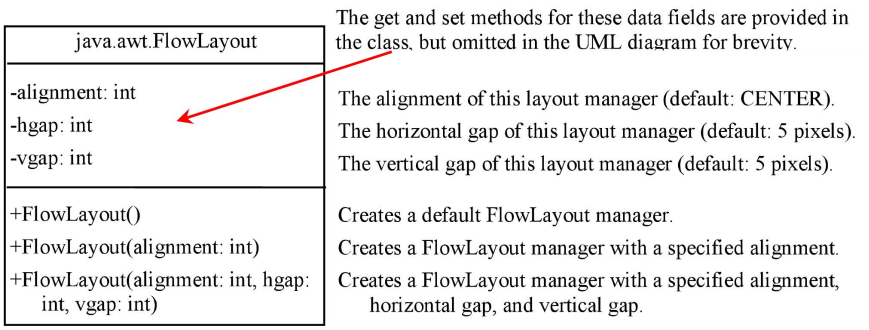
```
LayoutManager layoutManager = new FlowLayout();
container.setLayout(layoutManager);
                OR
container.setLayout(new FlowLayout());
```

Java supplies several layout managers including

FlowLayout: The Simplest layout manager (the default one)Components are placed left to right in the order in which they were added. When one row is filled, a new row is started. Components

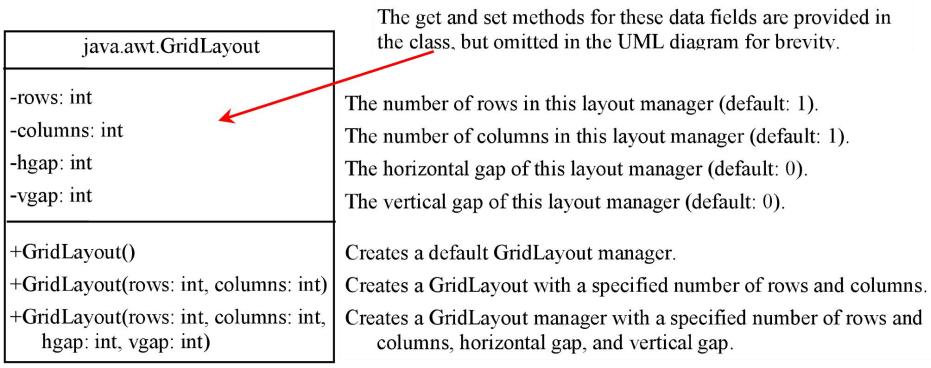
can be right aligned, centered or left aligned by using **FlowLayout.RIGHT**, **FlowLayout.CENTER**, and **FlowLayout.LEFT** respectivley.

FlowLayout has the following possible constructors:



GridLayout: The GridLayout manager divides the container up into a given number of rows and columns. All sections of the grid are equally sized and as large as possible.

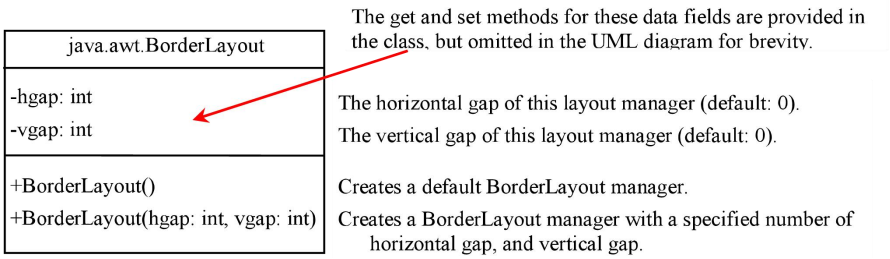
GridLayout can be set using one of the constructors depicted below:



BorderLayout: The BorderLayout manager divides the window into five areas: East, South, West, North, and Center. At most five components can be added. If you want more components, add a Panel, then add components to it. Components are added to a BorderLayout by using **add(Component, index)** where index is a constant such as :

- BorderLayout.EAST
- BorderLayout.SOUTH
- BorderLayout.WEST
- BorderLayout.NORTH
- BorderLayout.CENTER

BorderLayout can be set using one of the constructors shown below:



4. Attach Listeners to your components:- interacting with a component causes an Event to occur. Usually, we click button, move our mouse pointer onto a text, move our mouse pointer onto a button, enter a text to a text field and etc. When a certain action is performed, some event is fired. Listeners are associated with control so that it can sense whenever an event is fired.

Event Class	Listener Interface	Listener Methods	Example Source Object
ActionEvent	ActionListener	actionPerformed(ActionEvent)	JButton, JTextField
MouseEvent	MouseListener	mousePressed(MouseEvent)	Component
		mouseReleased(MouseEvent)	
		mouseEntered(MouseEvent)	
		mouseExited(MouseEvent)	
		mouseClicked(MouseEvent)	
WindowEvent	WindowListener	windowOPened(WindowEvent)	Window
		windowClosed(WindowEvent)	
		windowClosing(WindowEvent)	
		windowIconified(WindowEvent)	
		windowDeiconified(WindowEvent)	

Note that If a component can fire an event, any subclass of the component can fire the same type of event. For example, every GUI component can fire **MouseEvent**, **KeyEvent**, **FocusEvent**, and **ComponentEvent**, since **Component** is the superclass of all GUI components. Some of the packages of JDK API that we use in handling event-driven tasks include:

- java.awt.event.ActionEvent
- java.awt.event.ActionListener

- java.awt.event.MouseEvent
- java.awt.event.MouseListener
- java.awt.event.WindowEvent
- java.awt.event.WindowListener

Listeners, registrations, and handling of events:

Delegation-based model for event handling is employed where a source object fires an event, and an object interested in the event (a listener)handles the event. For an object to be a listener for an event on a source object, two things are needed. Firstly, the listener object must be an instance of the corresponding event-listener interface to ensure that the listener has the correct method for processing the event. Java provides a listener interface for every type of event. The listener interface is usually named **XListener** for **XEvent**, with the exception of **MouseMotionListener**. For example, the corresponding listener interface for **ActionEvent** is **ActionListener** and each listener for **ActionEvent** should implement the **ActionListener** interface. Secondly, the listener object must be registered by the source object. Registration methods depend on the event type. For **ActionEvent**, the method is **addActionListener**. In general, the method is named **addXListener** for **XEvent**. A source object may fire several types of events. It maintains, for each event, a list of registered listeners and notifies them by invoking the *handler* of the listener object to respond to the event

Example: Assume I have window/frame that has a label named "Name" and a text field to enter name. There is also a button of which text is "show me my input in dialog box". Once we enter a name in the text field, we can click the button to get a dialog box that contain the name we entered in the text field. How can we implement this scenario?

Solution: There could be different solutions. But, let us see one way of setting the layout.

First, let us create the necessary containers and components with appropriate layout; then, we will add the components/containers to the appropriate containers. Here we go!

```
import java.awt.GridLayout;
import javax.swing.*;
public class Example1 extends JFrame{
        JLabel label_name;
        JTextField txt_field_name;
        JButton btn_show;
        Example1(){
                label_name= new JLabel("Name:");
                txt_field_name = new JTextField();
                btn_show = new JButton("show me my input in dialog box");

        //I want to create a JPanel that could contain the label of the name and the text field of the
        //name and, then, I will add this JPanel instance to the JFrame
```

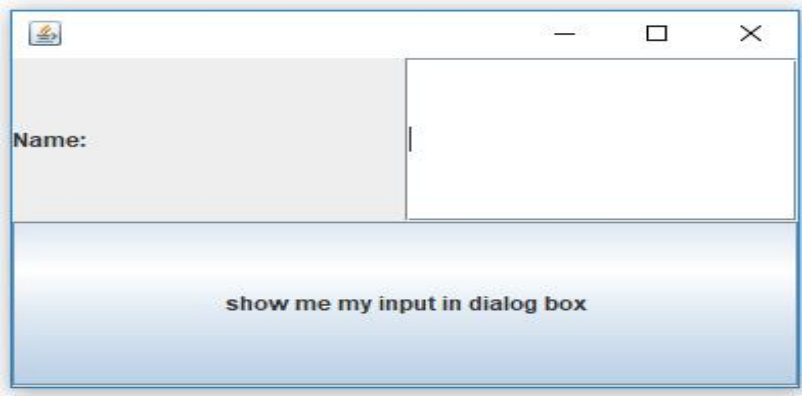
```
        JPanel panel1 = new JPanel();
        panel1.setLayout(new GridLayout(1,2));
        panel1.add(label_name);
        panel1.add(txt_field_name);

        //let us add the panel and the button to the frame after set the layout of the frame
        this.setLayout(new GridLayout(2,1));
        this.add(panel1);
        this.add(btn_show);

        //let us set the size, location and set default close aspects of the frame and make the
        window //visible
        this.setSize(400,250);
        this.setLocation(150,200);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setVisible(true);
}

public static void main(String [] args){
new Example1();
}
}
```

If we run the above code, we will get the following GUI application.



But, clicking the button does not do anything as we did not associate this action on the button with the appropriate event. Thus, we are going to improve this code so that clicking the button shows the desired dialog box.

We can do this using one of the following three ways. The first is register the event listener of the source object and define the action performed at the same spot or register the event listener on the source object and define action performed in another class. This second way can, in turn,

be accomplished in two ways: using an inner class for the action definition or using another separate class. Let us see the implementation using the three approaches.

Approach 1:

```
import java.awt.GridLayout;
import java.awt.event.*;
import javax.swing.*;

public class Example1 extends JFrame{
        JLabel label_name;
        JTextField txt_field_name;
        JButton btn_show;
Example1(){
                label_name= new JLabel("Name:");
                txt_field_name = new JTextField();
                btn_show = new JButton("show me my input in dialog box");
          btn_show.addActionListener(new ActionListener(){
            @Override
            public void actionPerformed(ActionEvent e) {
                  JOptionPane.showMessageDialog(null,"You entered
            name:"+txt_field_name.getText());
            }

          });
//I want to create a JPanel that could contain the label of the name and the text field of the //name and, then, I will add this JPanel instance to the JFrame

JPanel panel1 = new JPanel();
panel1.setLayout(new GridLayout(1,2));
panel1.add(label_name);
panel1.add(txt_field_name);

//let us add the panel and the button to the frame after set the layout of the frame
this.setLayout(new GridLayout(2,1));
this.add(panel1); this.add(btn_show);

//let us set the size, location and set default close aspects of the frame and make the window //visible
this.setSize(400,250);
this.setLocation(300,400);
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
this.setVisible(true);
}
public static void main(String [] args){
   //we are calling the JFrame instance
new Example1();
```

```
}
}
```

Approach 2:

```
import java.awt.GridLayout;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.JTextField;

public class Approach2 extends JFrame{
        JLabel label_name;
        JTextField txt_field_name;
        JButton btn_show;
Approach2(){
                label_name= new JLabel("Name:");
                txt_field_name = new JTextField();
                btn_show = new JButton("show me my input in dialog box");
          btn_show.addActionListener(new Listenn_btn_click());
//I want to create a JPanel that could contain the label of the name and the text field of the
//name and, then, I will add this JPanel instance to the JFrame

JPanel panel1 = new JPanel();
panel1.setLayout(new GridLayout(1,2));
panel1.add(label_name);
panel1.add(txt_field_name);

//let us add the panel and the button to the frame after set the layout of the frame
this.setLayout(new GridLayout(2,1));
this.add(panel1); this.add(btn_show);

//let us set the size, location and set default close aspects of the frame and make the
window //visible
this.setSize(400,250);
this.setLocation(300,400);
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
this.setVisible(true);
}
public static void main(String [] args){
   //we are calling the JFrame instance
new Approach2();
}
//This is a class that shows the action to be performed when the clicking event is fired
private class Listenn_btn_click implements ActionListener {
      @Override
      public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(null, "You entered name:
"+txt_field_name.getText());
```

```
            }
        }
        }
```
Approach 3:
```
import java.awt.GridLayout;
import java.awt.event.*;
import javax.swing.*;

public class Approach3 extends JFrame{
        JLabel label_name;
        static JTextField txt_field_name;
        JButton btn_show;
Approach3(){
                label_name= new JLabel("Name:");
                txt_field_name = new JTextField();
                btn_show = new JButton("show me my input in dialog box");
          btn_show.addActionListener(new Listenn_btn_click());
//I want to create a JPanel that could contain the label of the name and the text field of the //name
and, then, I will add this JPanel instance to the JFrame

JPanel panel1 = new JPanel();
panel1.setLayout(new GridLayout(1,2));
panel1.add(label_name);
panel1.add(txt_field_name);

//let us add the panel and the button to the frame after set the layout of the frame
this.setLayout(new GridLayout(2,1));
this.add(panel1); this.add(btn_show);

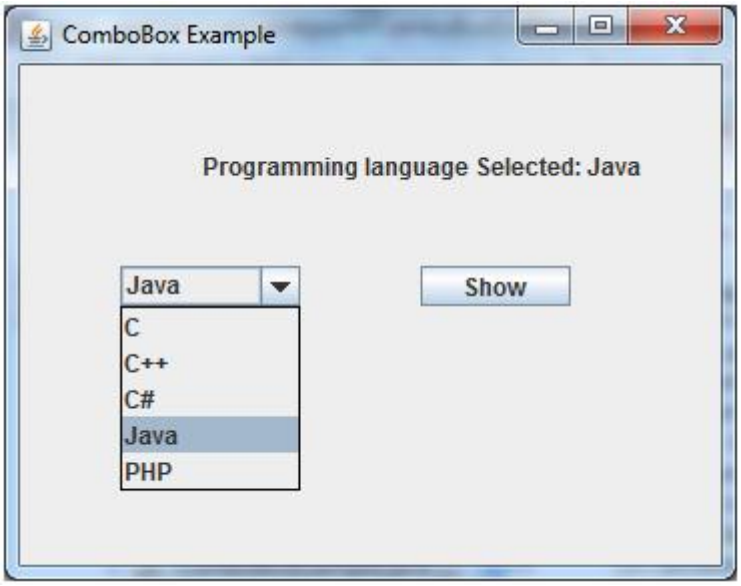
//let us set the size, location and set default close aspects of the frame and make the window
//visible
this.setSize(400,250);
this.setLocation(300,400);
this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
this.setVisible(true);
}
public static void main(String [] args){
   //we are calling the JFrame instance
new Approach3();
}
}
class Listenn_btn_click implements ActionListener {
      @Override
      public void actionPerformed(ActionEvent e) {
         JOptionPane.showMessageDialog(null, "You entered name:
"+Approach3.txt_field_name.getText());
```

```
    }
}
```

Java JComboBox Example with ActionListener

```
1. import javax.swing.*;
2. import java.awt.event.*;
3. public class ComboBoxExample {
4. JFrame f;
5. ComboBoxExample(){
6.     f=new JFrame("ComboBox Example");
7.     final JLabel label = new JLabel();
8.     label.setHorizontalAlignment(JLabel.CENTER);
9.     label.setSize(400,100);
10.    JButton b=new JButton("Show");
11.    b.setBounds(200,100,75,20);
12.    String languages[]={"C","C++","C#","Java","PHP"};
13.    final JComboBox cb=new JComboBox(languages);
14.    cb.setBounds(50, 100,90,20);
15.    f.add(cb); f.add(label); f.add(b);
16.    f.setLayout(null);
17.    f.setSize(350,350);
18.    f.setVisible(true);
19.    b.addActionListener(new ActionListener() {
20.        public void actionPerformed(ActionEvent e) {
21. String data = "Programming language Selected: "
22.    + cb.getItemAt(cb.getSelectedIndex());
23. label.setText(data);
24. }
25. });
26. }
27. public static void main(String[] args) {
28.    new ComboBoxExample();
29. }
30. }
```

Output:



Java JRadioButton Example with ActionListener

```
1. import javax.swing.*;
2. import java.awt.event.*;
3. class RadioButtonExample extends JFrame implements ActionListener{
4. JRadioButton rb1,rb2;
5. JButton b;
6. RadioButtonExample(){
7. rb1=new JRadioButton("Male");
8. rb1.setBounds(100,50,100,30);
9. rb2=new JRadioButton("Female");
10. rb2.setBounds(100,100,100,30);
11. ButtonGroup bg=new ButtonGroup();
12. bg.add(rb1);bg.add(rb2);
13. b=new JButton("click");
14. b.setBounds(100,150,80,30);
15. b.addActionListener(this);
16. add(rb1);add(rb2);add(b);
17. setSize(300,300);
18. setLayout(null);
19. setVisible(true);
20. }
21. public void actionPerformed(ActionEvent e){
22. if(rb1.isSelected()){
23. JOptionPane.showMessageDialog(this,"You are Male.");
24. }
25. if(rb2.isSelected()){
```

```
26. JOptionPane.showMessageDialog(this,"You are Female.");
27. }
28. }
29. public static void main(String args[]){
30. new RadioButtonExample();
31. }}
```

Output:

